

Advanced Fuzzing and Crash Analysis

© Richard Johnson 2019

rjohnson@fuzzing.io

Day 3

Day 3 - Agenda

- Program Analysis Primer
 - Abstract Syntax Trees
 - Intermediate Languages
 - Control Flow Graphs
 - Dataflow Graphs
- Program Analysis Applications
 - Taint Analysis
 - Symbolic Execution
- Browser Fuzzing

Program Analysis Applications

Code Transformation

- Most people writing plugins for compilers are concerned with code transformation since it is part of the normal operation of compilers
- We can use code transformation to add runtime analysis abilities
 - Execution trace
 - Code coverage
 - Data propagation
 - Determine indirect branch targets or code/data in binaries
 - Determine runtime generated values
 - libtokencap

Code Transformation

- Most people writing plugins for compilers are concerned with code transformation since it is part of the normal operation of compilers
- We can also use code transformation to add debugging helpers
 - LLVM Sanitizer Family
 - efence / libduma / libdislocator

Code Transformation

- Most people writing plugins for compilers are concerned with code transformation since it is part of the normal operation of compilers
- We can also use code transformation to add mitigations
 - FreeSentry – Use After Free mitigation from Cisco Talos

Code Transformation

- Most people writing plugins for compilers are concerned with code transformation since it is part of the normal operation of compilers
- We can also use code transformation to add fuzzing support
 - libFuzzer
 - afl-clang-fast

Threat Modeling

- Threat modeling relies on the ability to express untrusted boundaries and paths where untrusted data flows
- Reachability analysis shows the control flow paths from a source to a sink
- Properties can be enforced over those flows to apply constraints or checks for sanitization functions, etc

Code Coverage

- Code Coverage
 - Analysis of program runtime to determine execution flow
 - Collect the sequence of execution of basic blocks and branch edges
- Several approaches to collect dynamic branch traces
 - Native debugger API
 - CPU Branch Interrupts
 - Static binary rewriting
 - Dynamic binary instrumentation

Code Coverage

- Uses
 - Determine what code was executed across the whole process given an input
 - Determine which code executes uniquely across a set of inputs
 - Determine the code last executed before program exit
 - Locate paths for doing further path sensitive analysis
 - Augment incomplete static analysis with indirect branch information
 - Locate most exercised code to target for optimization

Taint Analysis

- Taint analysis is a dataflow analysis with user defined taint sources and sinks
- A taint violation occurs when untrusted input reaches critical locations
- Example: EIP=tainted

Symbolic Execution

- From SSA IL form, code is represented in single-instance variables and micro operations
- Micro operations can be expressed directly in Boolean algebra
- Symbolic execution is a form of emulation using Boolean algebra and constraints to interpret the outcome of code

Summary

- Program analysis techniques can be used to detect vulnerability classes or modify programs to assist with fuzzing
- Today we will focus on assisting fuzzing with code coverage, monitoring function call patterns, and building dictionaries with immediate values
- Plugins for source code analysis are mostly available, we will build those techniques for closed source binaries
- Tomorrow we will expand these techniques with taint analysis and symbolic execution

Program Analysis

What is Program Analysis

- Program Analysis is a series of techniques for abstracting and reasoning about programs
- Program Analysis is an automated dissection and abstraction of programs from source or binary to an abstracted model in order to prove a property of that program
- Can be static, dynamic or hybrid

What is Program Analysis

- There are many questions to be answered about programs
 - What conditions cause a program to exit
 - Does this parser contain memory safety issues
 - What is the format of the data the parser is capable of processing
 - At point P what are the possible values for variable V
 - What parts of the program have influence over a variable
 - What parts of the program deal with untrusted input

What is Program Analysis

- There are many applications for the answers
 - Type recovery
 - Decompilation
 - Bug hunting
 - Crash triage
 - Optimization
 - Refactoring
 - Instrumentation
 - Deobfuscation

What is Program Analysis

- Programs are:
 - Statements
 - Expressions
 - Types

What is Program Analysis

- Analyses can be categorized as path sensitive or insensitive
 - Type analysis is an example of path insensitive analysis
- To reason about security properties of a program, we need to consider the individual paths between two locations
- In order to do path sensitive analysis, graphs are used to represent the program

What is Program Analysis

- Rice's Theorem tells us many problems in program analysis are undecidable
- Approximations are made to create a computable program analysis algorithm
- Soundness is a measure of the ability to prove a property on a model
- Completeness is whether the model holds for all cases
 - A tool is complete if it produces no false positives

What is Program Analysis

- Programs are difficult to model and reason about
 - Loops
 - Indirection
 - Conditionals
 - Dynamic memory
 - Memory Aliasing
 - System environment
 - Exceptions and non-linear control flow
- Compromises in the soundness or completeness of analysis arise from these challenges

Type Analysis

- A foundational analysis is Type Analysis
- Types
 - Constant
 - Integer
 - Pointer
 - Function Pointers
- Constraints

Abstract Syntax Trees

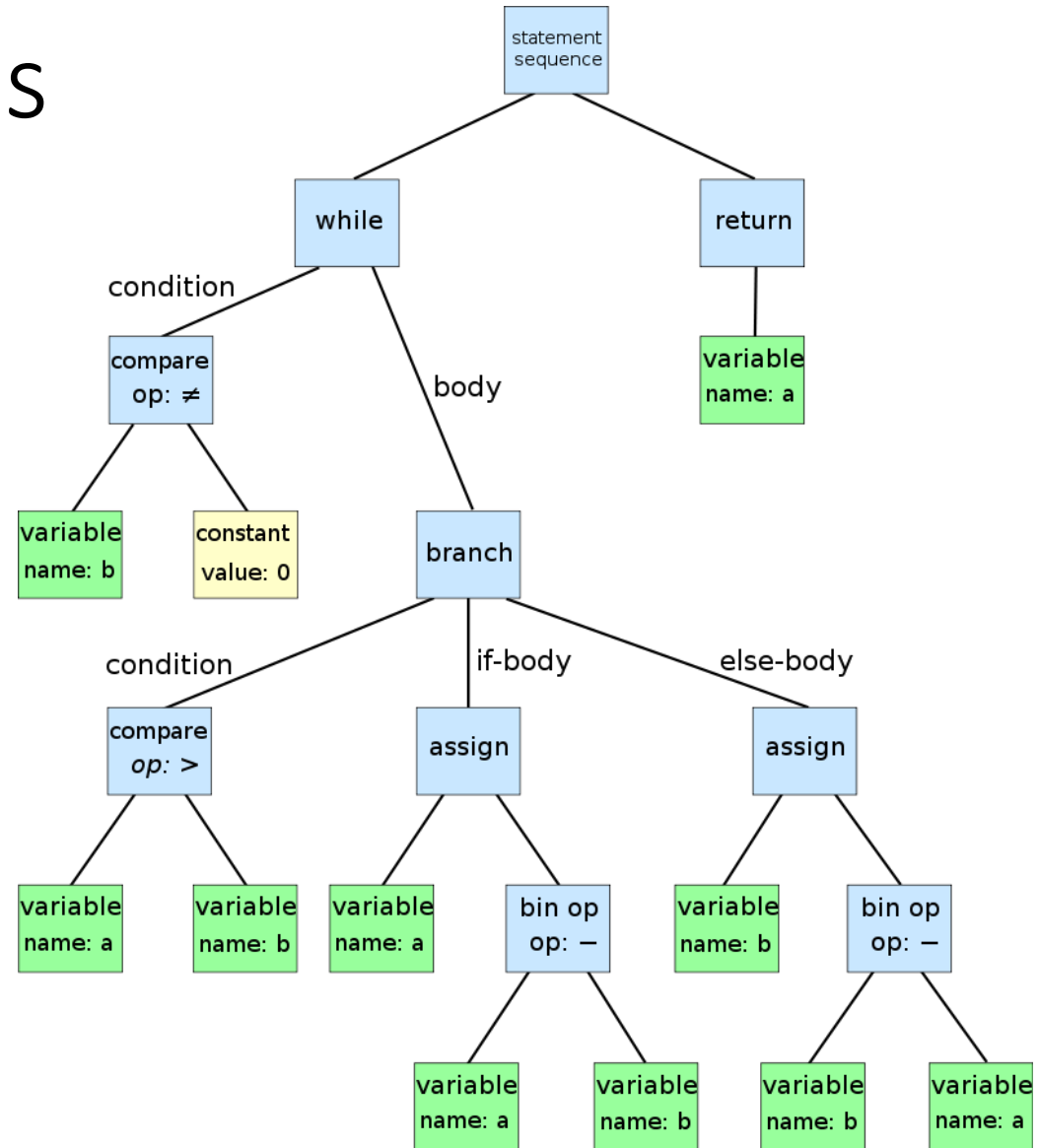
- Abstract Syntax Trees are the result of a syntax analysis by a compiler
- Parse Tree -> Abstract Syntax Tree
 - Parse Trees are concrete and represent everything in the source
 - Including punctuation and other syntactic sugar
 - ASTs create a canvas for doing the first phase of program analysis

Abstract Syntax Trees

- Variable types must be preserved, as well as the location of each declaration in source code.
- The order of executable statements must be explicitly represented and well defined.
- Left and right components of binary operations must be stored and correctly identified.
- Identifiers and their assigned values must be stored for assignment statements.

Abstract Syntax Trees

- \$ cat Euclidian.c
- while b $\neq$ 0
- if a > b
- a := a - b
- else
- b := b - a
- return a



Abstract Syntax Trees

- `$ cat hello.c`
- `include <stdio.h>`
- `int main(int argc, char **argv)`
- `{`
- `printf("Hello from %s\n", argv[0]);`
- `return 0;`
- `}`

Abstract Syntax Trees

- `$ clang -Xclang -ast-dump $1`
- ``-FunctionDecl 0x3900430 <hello.c:3:1, line:7:1> line:3:5 main 'int (int, char **).'`
- `|-ParmVarDecl 0x39002e0 <col:10, col:14> col:14 argc 'int'`
- `|-ParmVarDecl 0x3900358 <col:20, col:27> col:27 used argv 'char **'`
- ``-CompoundStmt 0x39006c8 <line:4:1, line:7:1>`
- `|-CallExpr 0x3900610 <line:5:2, col:35> 'int'`
- `-snip-`
- ``-ReturnStmt 0x39006b0 <line:6:2, col:9>`
- `-IntegerLiteral 0x3900690 <col:9> 'int' 0`

Abstract Syntax Trees

- | | -ImplicitCastExpr 0x39005f8 <col:2> 'int (*)(const char *, ...)'
<FunctionToPointerDecay>
- | | `DeclRefExpr 0x39004e8 <col:2> 'int (const char *, ...)' Function 0x38f2198 'printf'
'int (const char *, ...)'
- | | -ImplicitCastExpr 0x3900660 <col:9> 'const char *' <BitCast>
- | | `ImplicitCastExpr 0x3900648 <col:9> 'char *' <ArrayToPointerDecay>
- | | `StringLiteral 0x3900510 <col:9> 'char [15]' lvalue "Hello from %s\n"
- | `ImplicitCastExpr 0x3900678 <col:28, col:34> 'char *' <LValueToRValue>
- | `ArraySubscriptExpr 0x39005a8 <col:28, col:34> 'char *' lvalue
- | | -ImplicitCastExpr 0x3900590 <col:28> 'char **' <LValueToRValue>
- | | `DeclRefExpr 0x3900548 <col:28> 'char **' lvalue ParmVar 0x3900358 'argv'
'char **'
- | `IntegerLiteral 0x3900570 <col:33> 'int' 0

Intermediate Languages

- Intermediate languages fill the gap between AST and Native ISA
- Several languages can be used in different phases of optimization
- Different layers of abstraction allow reasoning about different properties

Intermediate Languages

- `$ clang -S -emit-llvm $1 -o -`
- `; ModuleID = 'hello.c'`
- `target datalayout = "e-m:e-i64:64-f80:128-n8:16:32:64-S128"`
- `target triple = "x86_64-pc-linux-gnu"`
- `@.str = private unnamed_addr constant [15 x i8] c"Hello from %s\0A\00", align 1`

Intermediate Languages

- ; Function Attrs: nounwind uwtable
- define i32 @main(i32 %argc, i8** %argv) #0 {
- %1 = alloca i32, align 4
- %2 = alloca i32, align 4
- %3 = alloca i8**, align 8
- store i32 0, i32* %1
- store i32 %argc, i32* %2, align 4
- store i8** %argv, i8*** %3, align 8
- %4 = load i8**, i8*** %3, align 8
- %5 = getelementptr inbounds i8*, i8** %4, i64 0
- %6 = load i8*, i8** %5, align 8
- %7 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([15 x i8], [15 x i8]* @.str, i32 0, i32 0), i8* %6)
- ret i32 0
- }

Intermediate Languages (BAP IR)

- `$ bap -dbir --print-symbol=main hello`
- `00000184: program`
- `00000127: sub main()`
- `000000f7:`
- `000000f8: v259 := RBP`
- `000000f9: RSP := RSP - 8`
- `000000fa: mem := mem with [RSP, el]:u64 <- v259`
- `000000fb: RBP := RSP`
- `000000fd: RSP := RSP - 0x20`
- `00000104: RAX := 0x4005A4`

Intermediate Languages (BAP IR)

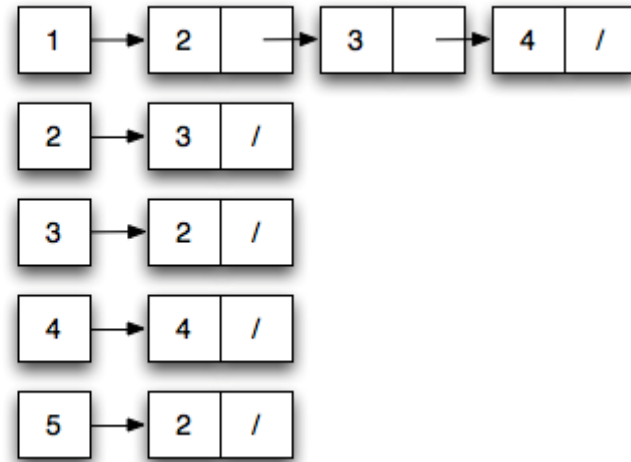
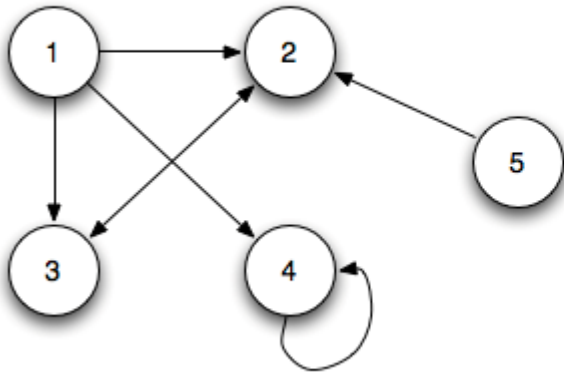
- 00000105: mem := mem with [RBP + 0xFFFFFFFFFFFFFFFFFC, el]:u32 <- 0
- 00000106: mem := mem with [RBP + 0xFFFFFFFFFFFFFFFF8, el]:u32 <- low:32[RDI]
- 00000107: mem := mem with [RBP + 0xFFFFFFFFFFFFFFFF0, el]:u64 <- RSI
- 00000108: RSI := mem[RBP + 0xFFFFFFFFFFFFFFFF0, el]:u64
- 00000109: RSI := mem[RSI, el]:u64
- 0000010a: RDI := RAX
- 0000010b: RAX := high:56[RAX].0
- 0000010c: RSP := RSP - 8
- 0000010d: mem := mem with [RSP, el]:u64 <- 0x400511
- 0000010e: call @printf with return %0000010f

Intermediate Languages (BAP IR)

- 0000010f:
- 00000110: RCX := pad:64[0]
- 00000117: mem := mem with [RBP + 0xFFFFFFFFFFFFFFEC, el]:u32 <- low:32[RAX]
- 00000118: RAX := pad:64[low:32[RCX]]
- 0000011b: RSP := RSP + 0x20
- 00000122: RBP := mem[RSP, el]:u64
- 00000123: RSP := RSP + 8
- 00000124: v229 := mem[RSP, el]:u64
- 00000125: RSP := RSP + 8
- 00000126: return v229

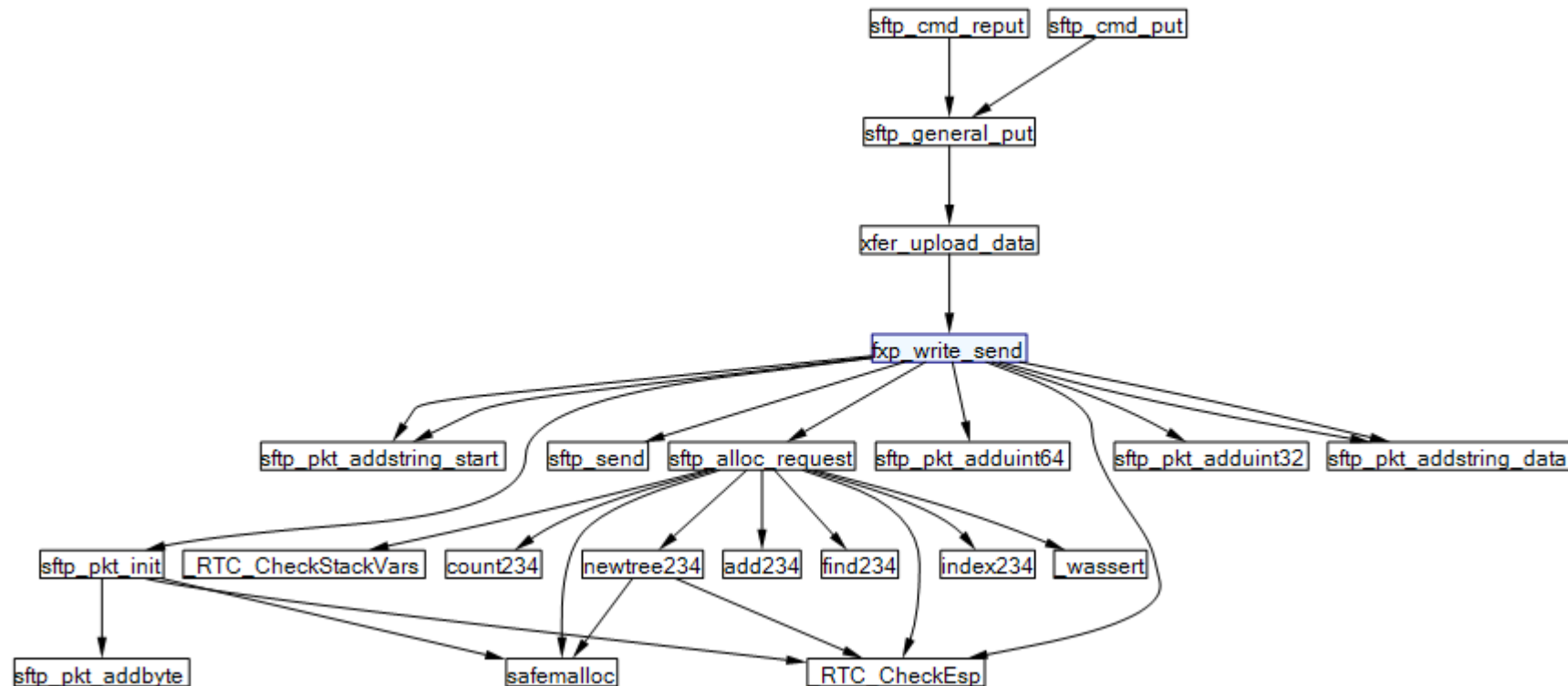
Graphs

- $G = (V, E)$
 - A graph is a set of vertices and a set of edges
 - Code is directional and hierarchical so directed graphs are used
 - Sparse directed graphs should be stored in an Adjacency List



Call Graphs

- A Call Graph is a representation of control flow between functions



Call Graphs

- Traditional call graph generation

Collect all call edges

```
foreach(Function in ModuleFunctions)
  foreach(CallInstruction in Instructions)
    AddCallEdge(Function, CallTarget))
```

Find edges for Function

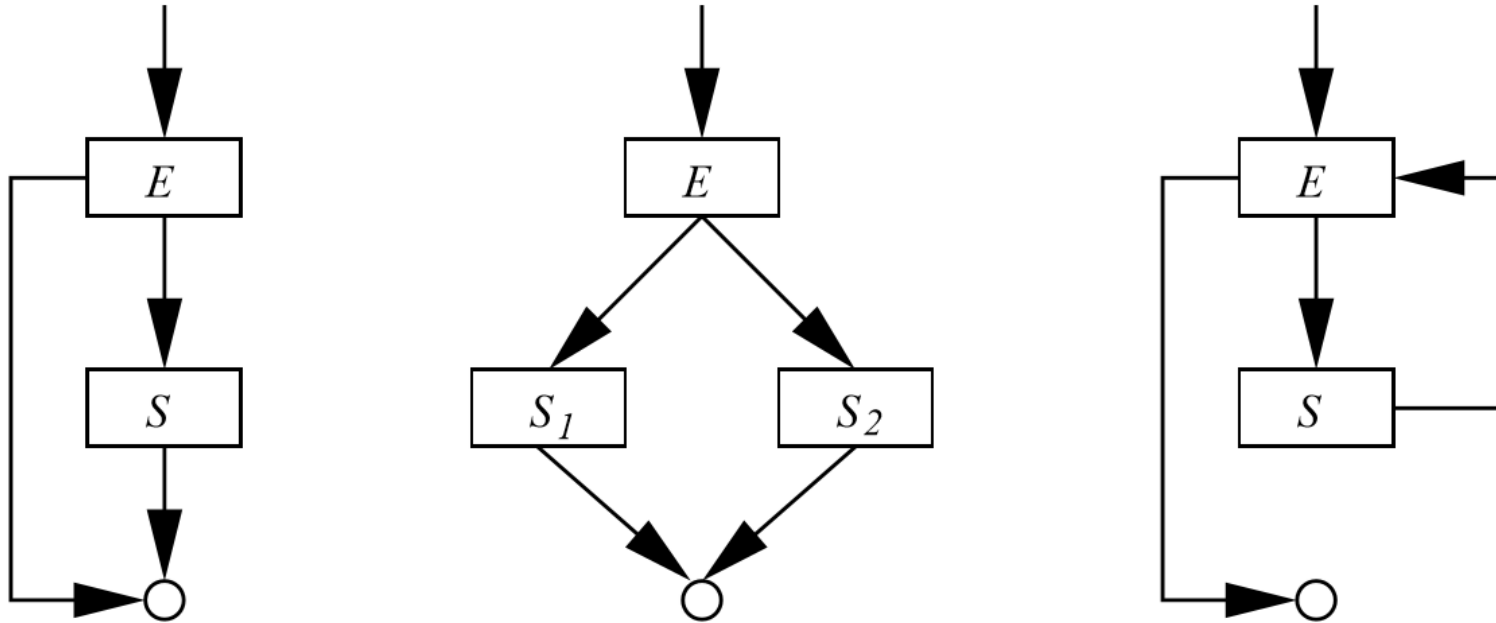
```
foreach(Edge in CallEdges)
  if(Target == Function)
    EdgesTo.Add(Edge)
  if(Source == Function)
    EdgesFrom.Add(Edge)
```

Control Flow Graphs

- A control flow graph (CFG) is a directed graph, in which nodes correspond to program points and edges represent possible flow of control.
- A CFG always has a single point of entry, denoted entry, and a single point of exit, denoted exit.
- If v is a node in a CFG then $\text{pred}(v)$ denotes the set of predecessor nodes and $\text{succ}(v)$ the set of successor nodes.

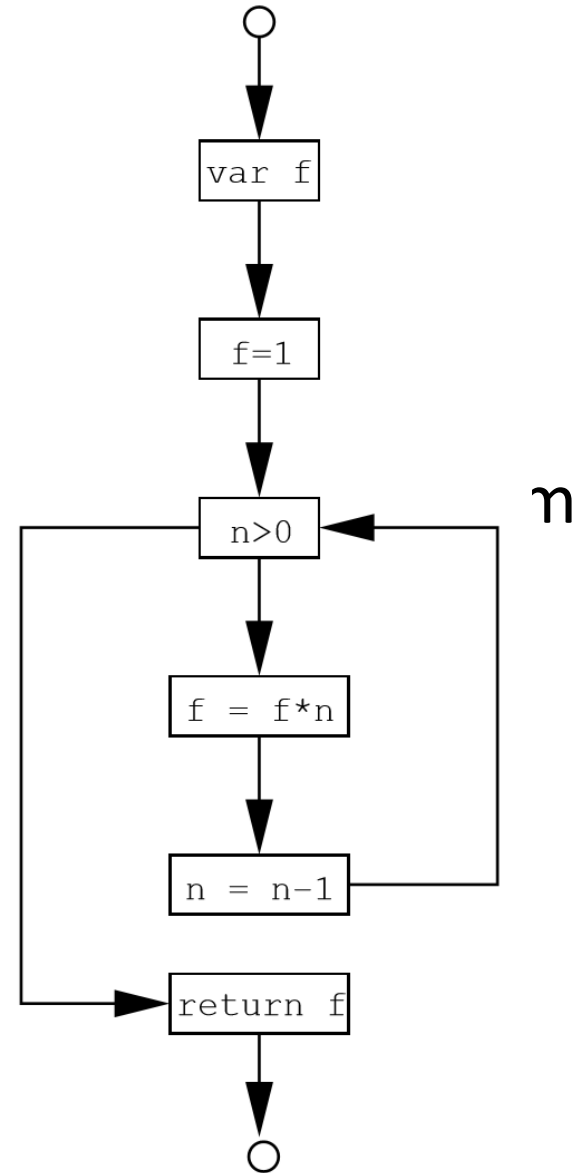
Control Flow Graphs

- CFGs represent program logic in an intuitive abstract graph



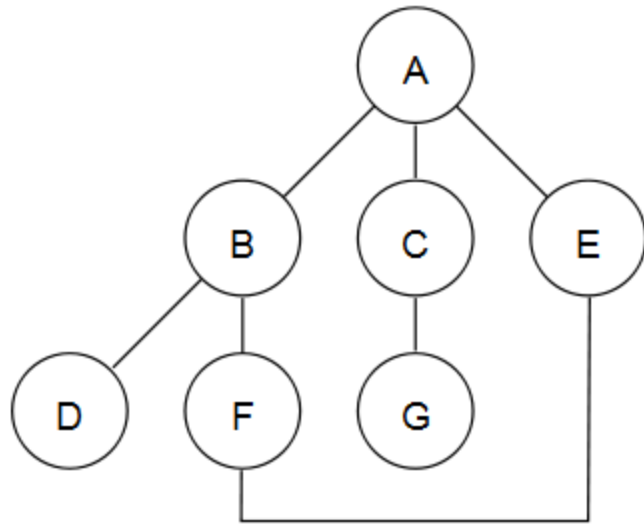
Control Flow Graphs

- CFGs represent program logic in an intuitive way
- This graph represents all possible paths in a program



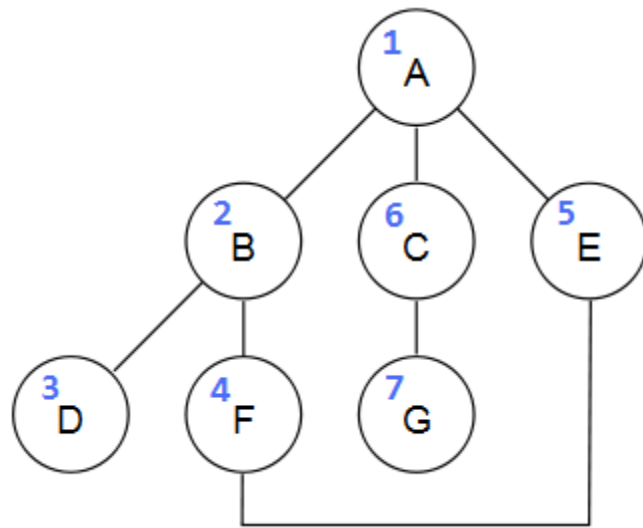
Graph Traversal

- Depth First Search
 - Visit nodes following edges as deep as possible before returning to the next edge



Graph Traversal

- Depth First Search
 - Visit nodes following edges as deep as possible before returning to the next edge



Dataflow Analysis

- Dataflow analysis deals with the constraints and influence on variables in a program

Reaching Definitions

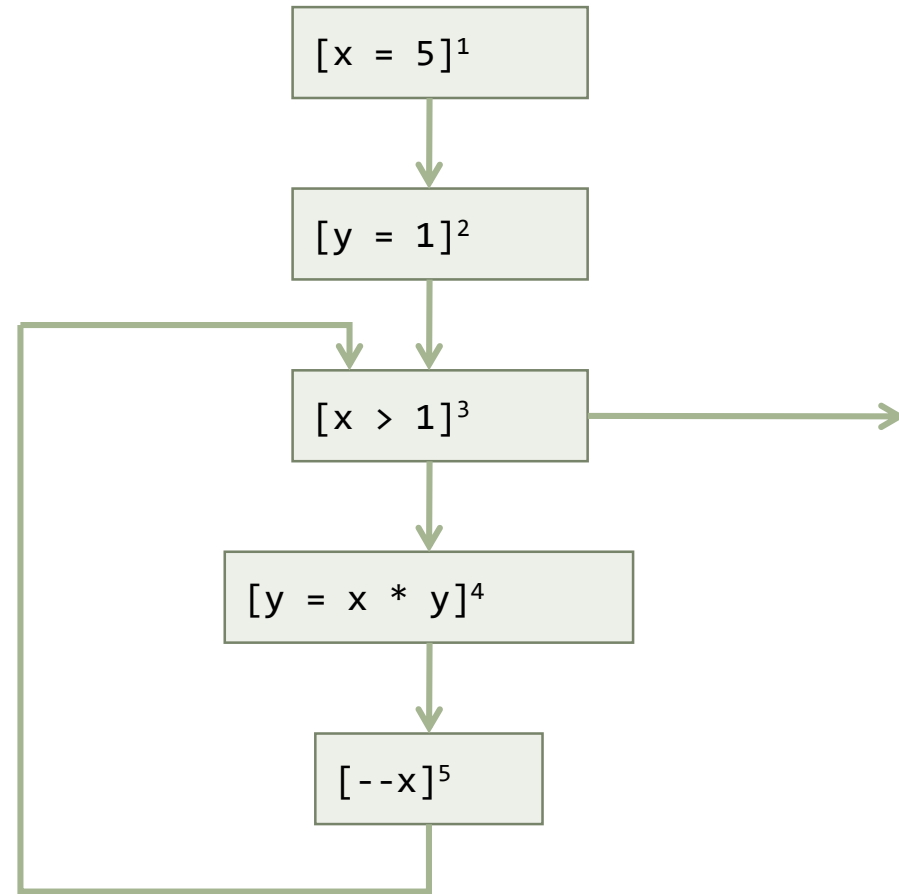
- A variable assignment $[x := a]$ may reach a code location if there is an execution of the program where x was last assigned at l when the code location is reached
- An assignment reaches the entry of a block if it reaches the exit of any of the blocks that precede it

Reaching Definitions

```
x = 5;  
y = 1;  
while (x > 1)  
{  
    y = x * y;  
    --x;  
}
```

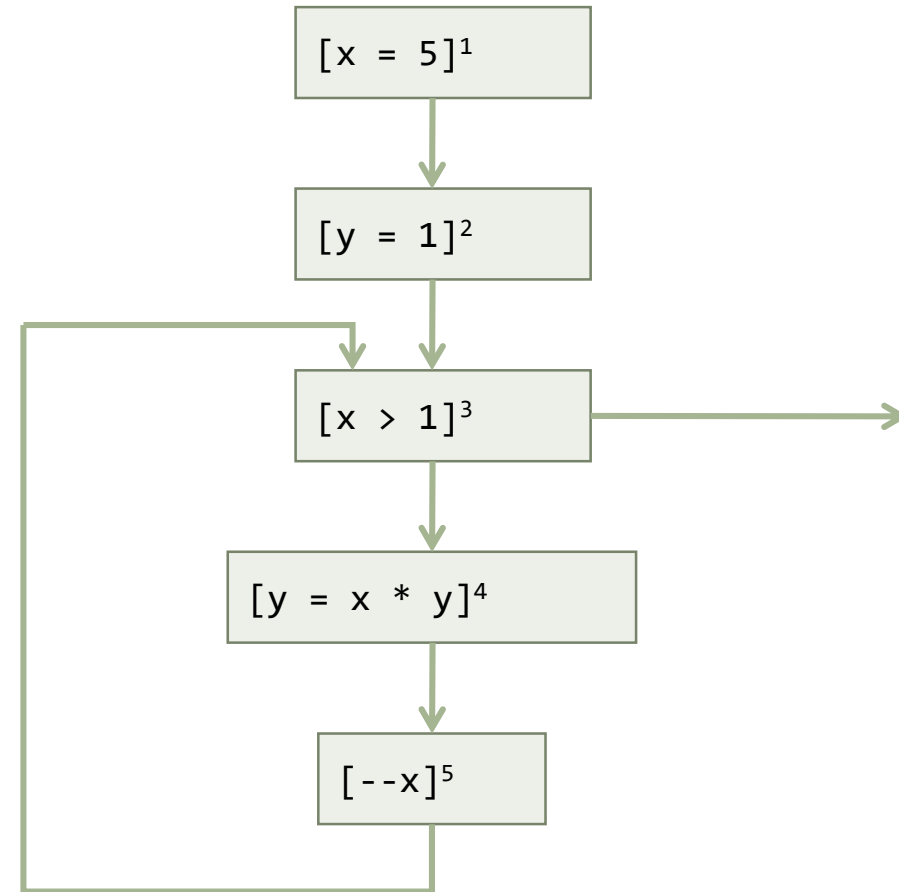
Reaching Definitions

```
x = 5;  
y = 1;  
while (x > 1)  
{  
    y = x * y;  
    --x;  
}
```



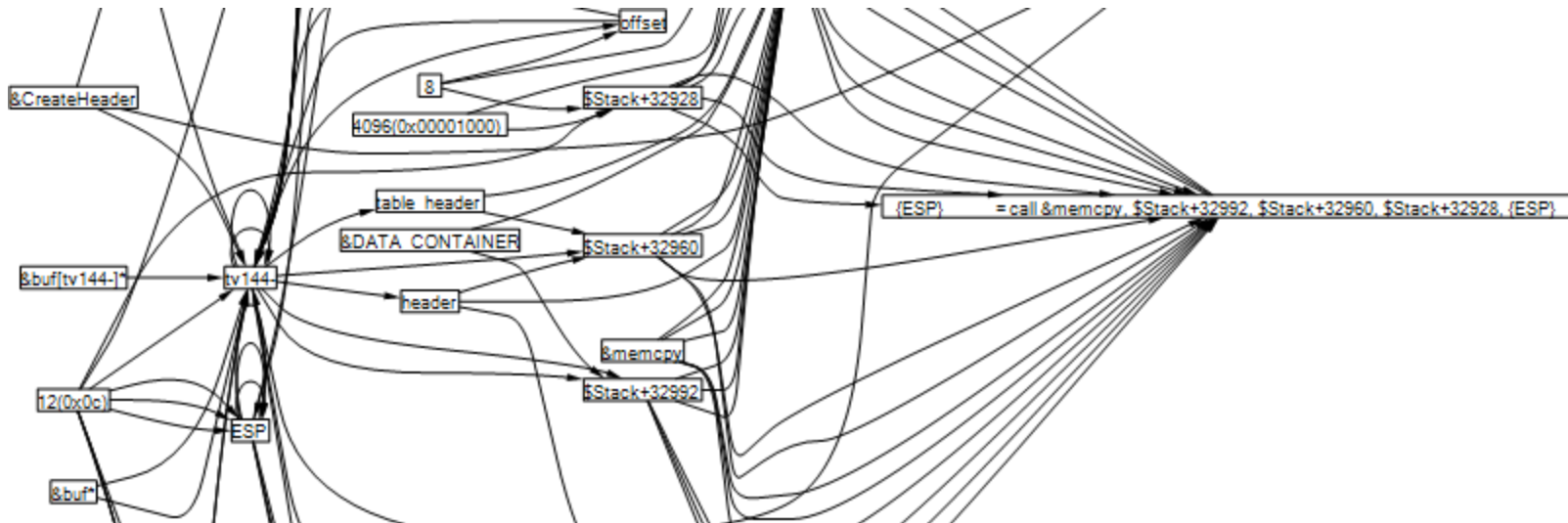
Reaching Definitions

	RD_{entry}	RD_{exit}
1	$(x, ?), (y, ?)$	$(x, 1), (y, ?)$
2	$(x, 1), (y, ?)$	$(x, 1), (y, 2)$
3	$(x, 1), (x, 5), (y, 2)$	$(x, 1), (x, 5), (y, 2)$
4	$(x, 1), (x, 5), (y, 2), (y, 4)$	$(x, 1), (x, 5), (y, 4)$
5	$(x, 1), (x, 5), (y, 4)$	$(x, 5), (y, 4)$



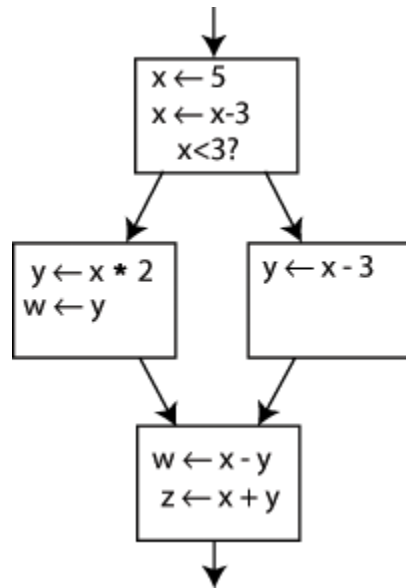
Data Flow Graphs

- Reaching definitions allow the construction of a context-free data flow graph



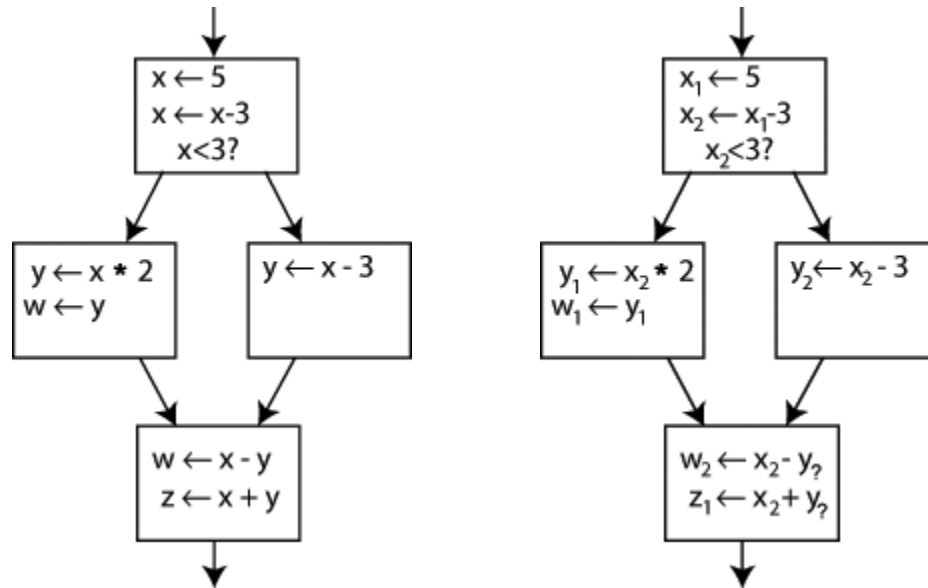
Single Static Assignment

- Intermediate form used by several compilers in which every variable is assigned only once



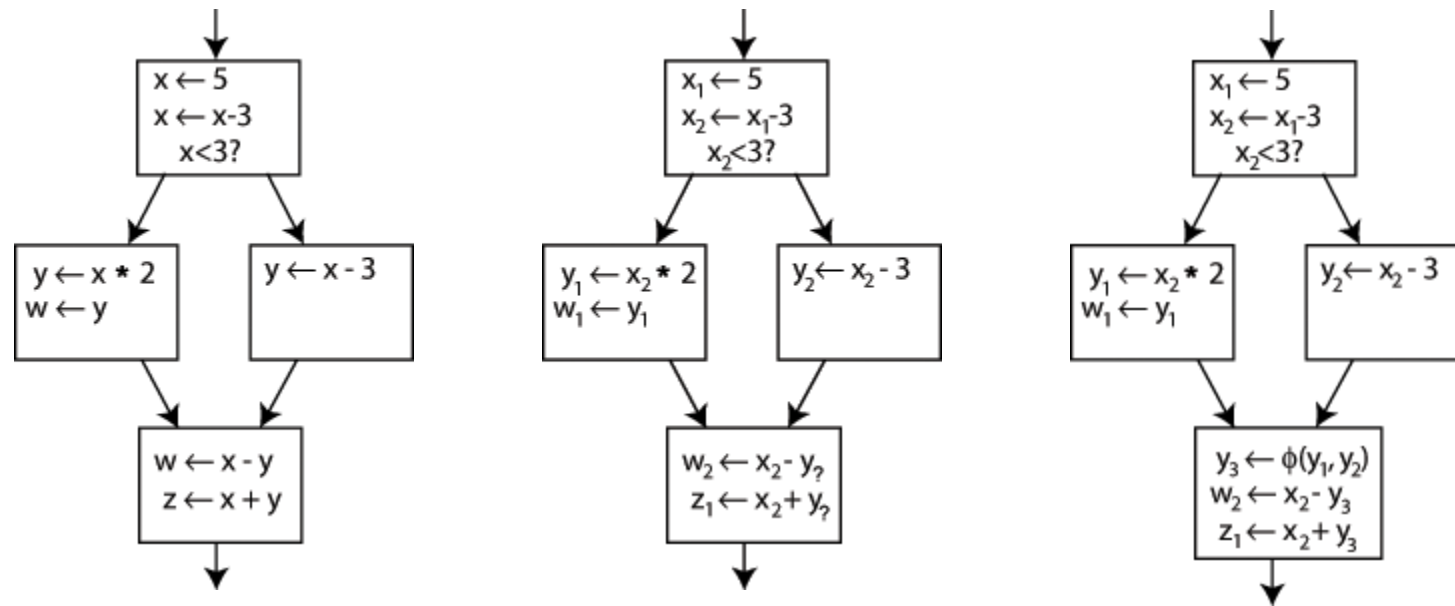
Single Static Assignment

- Use-definition relationships explicit
 - Each use reached by only one definition
 - Each definition dominates all uses



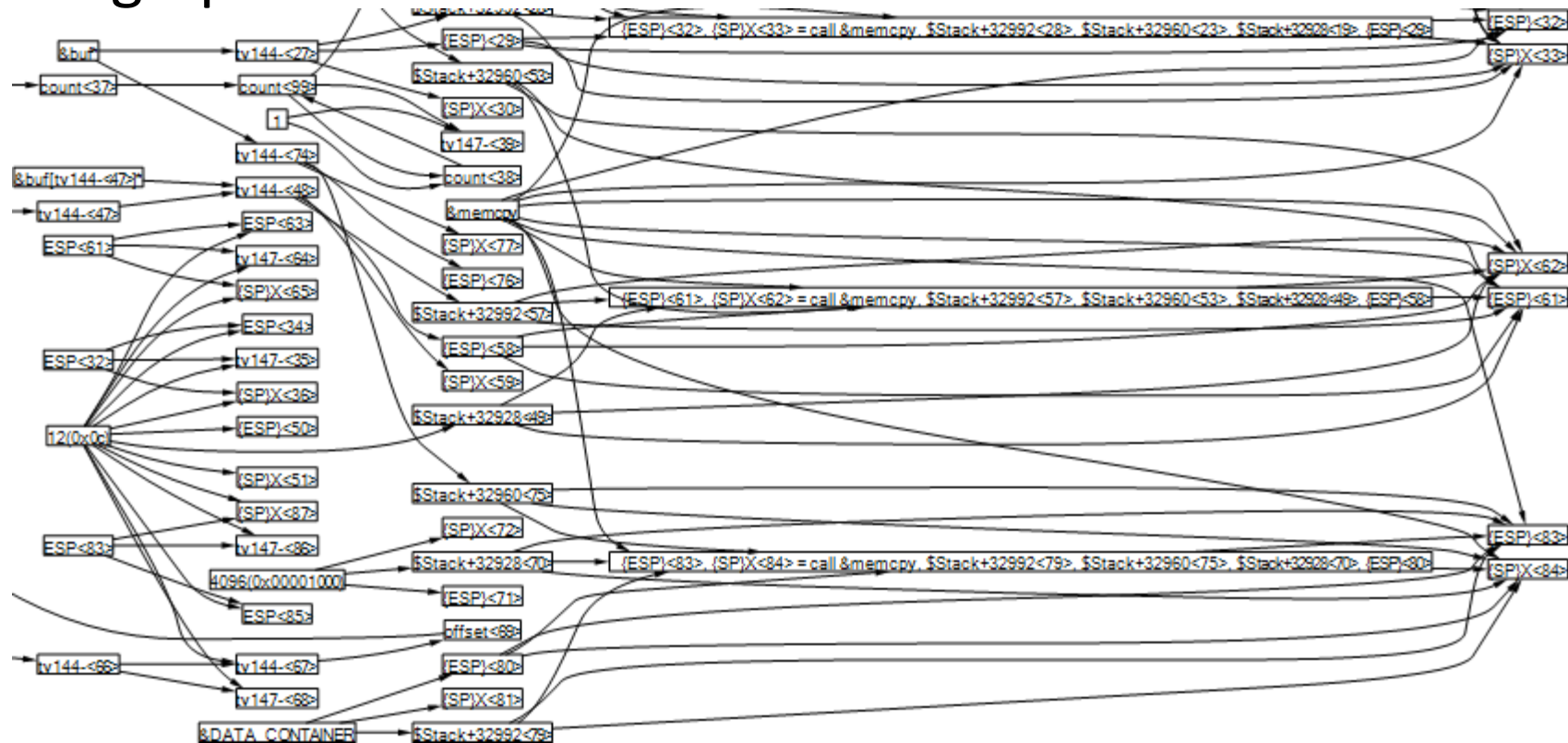
Single Static Assignment

- Special Φ (Phi) instructions are added to the beginning of blocks to represent joins of different versions of the same variable



Data Flow Graphs

- Converting to SSA form allows the simple construction of a contextual data flow graph



Dataflow Analysis

- Liveness Analysis
- A variable is live at a particular point in the program if its value at that point will be used in the future, otherwise it is dead
- Traditionally this is used for register allocation in a compiler but we can use it for security properties as well

Dataflow Analysis

- Liveness Analysis **Algorithm**

```
for each node  $n$  in CFG  
     $\text{in}[n] = \emptyset; \text{out}[n] = \emptyset$   
repeat  
    for each node  $n$  in CFG  
         $\text{in}'[n] = \text{in}[n]$   
         $\text{out}'[n] = \text{out}[n]$   
         $\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$   
         $\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$   
    until  $\text{in}'[n] = \text{in}[n]$  and  $\text{out}'[n] = \text{out}[n]$  for all  $n$ 
```

} initialize solutions

} save current results

} solve data-flow equations

} test for convergence

This is **iterative data-flow analysis** (for liveness analysis)

Dataflow Analysis

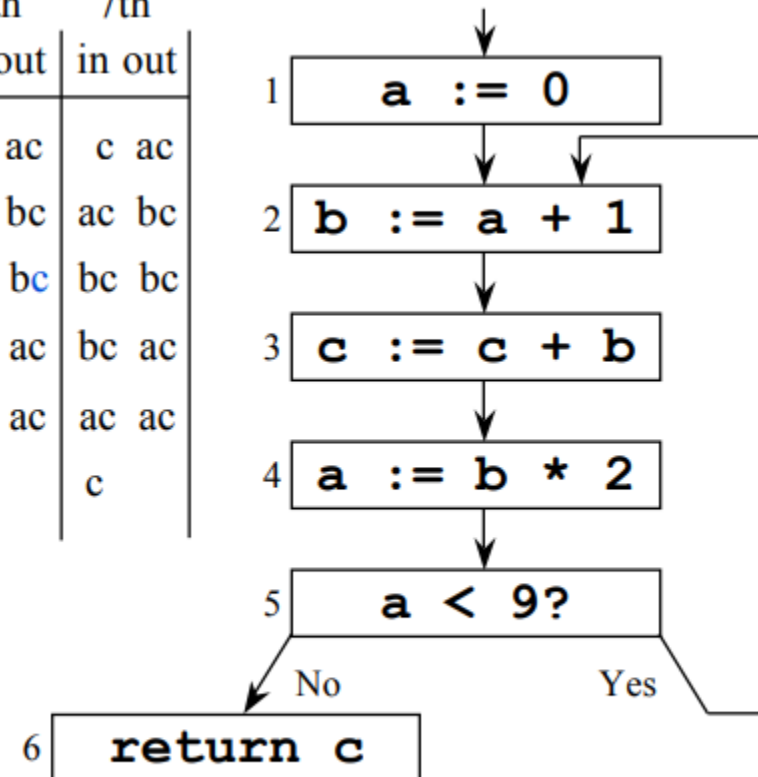
- Liveness Analysis

node #	use	def	1st		2nd		3rd		4th		5th		6th		7th	
			in	out	in	out	in	out	in	out	in	out	in	out	in	out
1		a			a		a		ac		c	ac	c	ac	c	ac
2	a	b	a		a	bc	ac	bc	ac	bc	ac	bc	ac	bc	ac	bc
3	bc	c	bc		bc	b	bc	b	bc	b	bc	b	bc	bc	bc	bc
4	b	a	b		b	a	b	a	b	ac	bc	ac	bc	ac	bc	ac
5	a		a	a	a	ac	ac	ac	ac	ac	ac	ac	ac	ac	ac	ac
6	c		c		c		c		c		c		c		c	

Data-flow Equations for Liveness

$$\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$$

$$\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$$



Dataflow Analysis

- Initialized Variable Analysis
- Given the use-def chain and the control flow path for a value, we can also check whether there was a definition of the value before it was used
- Compilers already do this for you!
 - If you know how to use them..

Dataflow Analysis

- `$ cat uninit.c`
- `#include <stdlib.h>`
- `#include <stdio.h>`
- `int main(int argc, char **argv)`
- `{`
- `int size;`
- `char *buf = malloc(size);`
- `printf("Hello from %s\n", buf);`
- `return 0;`
- `}`

Dataflow Analysis

- `$ clang -o uninit uninit.c`
- `$ ./uninit`
- Hello from
- `$ clang --analyze -o uninit uninit.c`
- `uninit.c:7:14: warning: Function call argument is an uninitialized value`
- `char *buf = malloc(size);`
- `^~~~~~`
- 1 warning generated.

Uninitialized Variables

- Intraprocedural analysis of uninitialized variables can also explicit if code is converted to SSA
- Unique variable labels are assigned on each write to a register or memory location
- If the zero instance of a variable is read from then uninitialized access has occurred

Uninitialized Variables

- `$ gdb -q -ex "disas main" -ex "q" uninit`
- Dump of assembler code for function main:
 - `0x0804844d <+0>: push ebp`
 - `0x0804844e <+1>: mov ebp,esp`
 - `0x08048450 <+3>: and esp,0xffffffff0`
 - `0x08048453 <+6>: sub esp,0x20`
 - `0x08048456 <+9>: mov eax,DWORD PTR [esp+0x18]`
 - `0x0804845a <+13>: mov DWORD PTR [esp],eax`
 - `0x0804845d <+16>: call 0x8048320 <malloc@plt>`

Uninitialized Variables

- `$ bap -dbir --print-symbol=main --ssa uninit`
- ...
- `00000121: v200.1 := EBP`
- `00000122: ESP.1 := ESP - 4`
- `00000123: mem.1 := mem with [ESP.1, el]:u32 <- v200.1`
- `00000124: EBP.1 := ESP.1`
- `00000125: ESP.2 := ESP.1 & 0xFFFFFFFF0`
- `0000012d: ESP.3 := ESP.2 - 0x20`
- `00000134: EAX.1 := mem.1[ESP.3 + 0x18, el]:u32`
- `00000135: mem.2 := mem.1 with [ESP.3, el]:u32 <- EAX.1`
- `00000136: ESP.4 := ESP.3 - 4`
- `00000137: mem.3 := mem.2 with [ESP.4, el]:u32 <- 0x8048462`
- `00000138: call @malloc with return %00000139`

Taint Driven Crash Analysis

Root Cause Analysis

- Execution Path
 - What code paths were executed
 - What parts of the execution interacted with external data
- Input Determination
 - Which input bytes influence the crash
- Root Cause
 - Which line of code needs to be patched

Common Vulnerability Analysis Scenarios

- Fuzzing
 - Spray 'n Pray
 - Grammar-based
 - “Fuzzing with Code Fragments”
- Static Analysis
 - Intra-procedural Analysis
 - Manual code review
- Third Party
 - In-the-wild exploitation
 - Vulnerability response teams
 - Vulnerability brokers

Previous Tooling

- Execution Path
 - DrCov, Process Stalker, CoverIt (hexblog), BlockCov, IDA PIN Block Trace
 - Bitblaze, Taintgrind, VDT
- Input Determination
 - delta, tmin, diff
- Exploitability
 - !exploitable
 - CrashWrangler
 - CERT Triage Tools

Automation Methods

- Execution Path
 - Code Coverage
 - Taint Analysis
- Input Determination
 - Slicing
- Exploitability
 - Symbolic Execution
 - Abstract Interpretation

Automation Methods

- Execution Path
 - Code Coverage
 - **Taint Analysis**
- Input Determination
 - **Slicing**

Taint Analysis

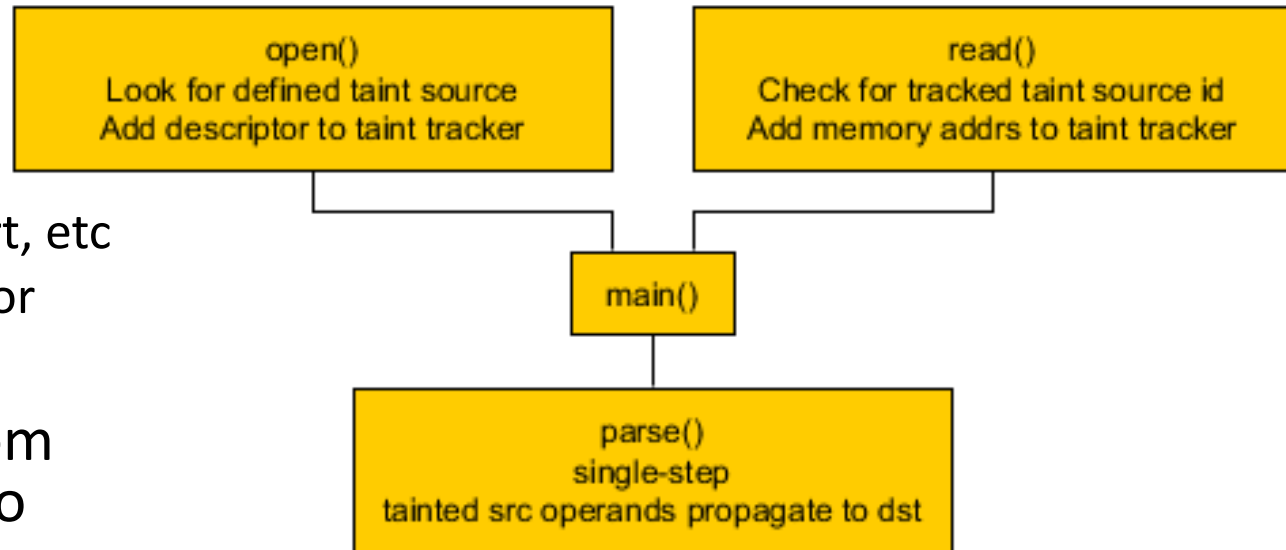
Concept

- Formally – Information Flow Analysis
 - Type of dataflow analysis
 - Can be static or dynamic, often hybrid
 - Applied to track user controlled data through execution
- Methodology
 - Define taint sources
 - Single-step execution
 - Apply taint propagation policy for each instruction
 - Apply taint checks (if any)

Concept

- Define Taint Sources

- Hook I/O Functions
- Look for taint sources
 - File name, network ip:port, etc
 - Track tainted file descriptor
- Single-step
- Add future data reads from taint source descriptors to the taint tracking engine
- Apply taint policy on each instruction



Concept

- Define Taint Sources
 - Hook I/O Functions
 - Look for taint sources
 - File name, network ip:port, etc
 - Track tainted file descriptor
 - Single-step
 - Add future data reads from taint source descriptors to the taint tracking engine
 - Apply taint policy on each instruction

EXPLICIT TAINT PROPAGATION

```
A = Taint()  
B = A  
C = B + 1  
D = C * B  
E = *(D)
```

IMPLICIT TAINT PROPAGATION

```
A = Taint()  
IF A > B:  
    C = TRUE  
ELSE:  
    C = FALSE
```


Implementation Details

- We will utilize Intel PIN to perform dynamic binary translation to instrument a target binary for tracing
- Binary translation is a robust program modification technique
 - JIT for hardware ISAs
- PIN supplies a robust API and framework for binary instrumentation
 - Supports easily hooking I/O functions for taint sources
 - High performance single-stepping
 - Supports instrumenting at instruction level for taint propagation / checks

Implementation Details

- We need to look for user defined taint sources returned from system calls and then single step watching memory propagate throughout the program
- To achieve our taint tracing we forked the PIN tool from the Binary Analysis Platform from Carnegie-Mellon University
 - Worked with the authors of BAP since early 2012 to improve the tracer so it performs acceptably against complex COTS software targets on Windows
 - Added code coverage and memory dump collection

Implementation Details – Trace Format

- The log is saved as a custom binary format with embedded protobuf structures. We use 'piqi' to define the structures and compile to protobuf

```
root@d14e0cf39781:/moflow/bap/libtracewrap/libtrace/piqi-files# cat  
stdframe.piqi
```

```
.include [  
    .module types  
]
```

```
% Frame representing the execution of a single assembly instruction  
.record [  
    .type
```

Implementation Details – Trace Format

```
% Address of the instruction  
.field [  
    .type address  
    .code 1  
]
```

```
% Thread id that executed the instruction  
.field [  
    .type thread-id  
    .code 2  
]
```

Implementation Details – Trace Format

```
% Raw bytes of the instruction
```

```
.field [
```

```
    .name rawbytes
```

```
    .type binary
```

```
    .code 3
```

```
]
```

Implementation Details – Trace Format

% Operands values read by the instruction

.field [

.name operand-pre-list

.type operand-value-list

.code 4

]

Implementation Details – Trace Format

```
% Operands values written by the instruction
.field [
    .name operand-post-list
    .type operand-value-list
    .code 5
    .optional
]
```

Implementation Details

- Taint Propagation Policy
 - Tree of tainted references to registers and bytes of memory are individually tracked
 - If input operands contain taint, propagate to all output operands
 - No control flow tainting
 - Optionally taint index registers
 - All index registers for LEA instructions are tainted
- No support for MMX, Floating point FCMOV, SSE PREFETCH

Design Considerations

- Taint Policy
 - Implicit Information Flows
 - Over-tainting
 - Most common when applying implicit taint via control flow
 - Under-tainting
 - If control flow taint is ignored
- Performance
 - Execution Speed
 - Analysis on each instruction is expensive
 - Avoid context switching
 - Memory Overhead

Taint Tracer Usage

```
root@d14e0cf39781:/moflow/slicer# ../bap/pin/pin -t ../tracer/gentrace32.so -help --  
./demo/demo
```

Pin tools switches

-bb_file [default]

Store the set of visited BBs in a text file

-coverage_track [default false]

Enable coverage tracking (only unique BBs)

-exn_file [default]

Store info about exception in a text file

-follow_progs

Follow the given program names if they are exec'd

Taint Tracer Usage

-freq [default 10000]

Report value of eip every n instructions.

-log_key_frames [default false]

Periodically output key frames containing important program values

-log_limit [default 0]

Number of instructions to limit logging to.

-log_syscalls [default false]

Log system calls (even those unrelated to taint)

Taint Tracer Usage

-logall_after [default false]

Log all (even untainted) instructions after the first tainted instruction

-logall_before [default false]

Log all (even untainted) instructions before and after the first tainted instruction

-logfile [default pintool.log]

The log file path and file name

-logone_after [default false]

Log the first instruction outside of the log range (taint-start/end), and then exit.

Taint Tracer Usage

-o [default out.bpt]

Trace file to output to.

-skip_taints [default 0]

Skip this many taint introductions

-snapshot_file [default]

File name for memory snapshot

-stack_dump [default 0]

How many bytes of stack to dump on exception

Taint Tracer Usage

-taint_args [default false]

Command-line arguments will be considered tainted

-taint_env

Environment variables to be considered tainted

-taint_files

Consider the given files as being tainted

-taint_indices [default false]

Values loaded with tainted memory indices will be considered tainted

-taint_net [default false]

Everything read from network sockets will be considered tainted

Taint Tracer Usage

-trig_count [default 0]

Number of times trigger will be executed before activating.

-trig_mod [default]

Module that trigger point is in.

-unique_logfile [default 0]

The log file names will contain the pid

-visited_bb_file [default]

Do not log hits to BBs listed in this file

Taint Tracer Usage

-trig_count [default 0]

Number of times trigger will be executed before activating.

-trig_mod [default]

Module that trigger point is in.

-unique_logfile [default 0]

The log file names will contain the pid

-visited_bb_file [default]

Do not log hits to BBs listed in this file


```
root@d14e0cf39781:/moflow/slicer# sudo ../bap/pin/pin -t ../tracer/gentrace32.so -taint
_indices -taint_files input.txt -snapshot-file /tmp/demo.snapshot -o /tmp/demo.trace --
../demo/demo tlv demo/input.txt 2>&1 | head -18
Logging initially enabled: 0
Code cache limit is 0
Starting program
This is modload()
This is modload()
Thread 0 starting
Not opening /etc/ld.so.cache
Not opening /lib32/libc.so.6
This is modload()
Opening tainted file: demo/input.txt
Tainting 1000 bytes from read at dda0a000, fd=5
Taint introduction #0. @dda0a000/1000 bytes: file demo/input.txt
adding new mapping from file demo/input.txt to 0 on taint num 1
adding new mapping from file demo/input.txt to 1 on taint num 2
adding new mapping from file demo/input.txt to 2 on taint num 3
adding new mapping from file demo/input.txt to 3 on taint num 4
adding new mapping from file demo/input.txt to 4 on taint num 5
adding new mapping from file demo/input.txt to 5 on taint num 6
```

```
adding new mapping from file demo/input.txt to ffd on taint num ffe  
adding new mapping from file demo/input.txt to ffe on taint num fff  
adding new mapping from file demo/input.txt to fff on taint num 1000
```

```
Activating taint analysis
```

```
First logged instruction
```

```
First tainted instruction
```

```
closed tainted fd 5
```

```
total_size: 65535, record_count 1111638594
```

```
----- Tainted Regs -----
```

```
ecx = ffffffff
```

```
eflags = ffffffff
```

```
ymm0 = ffffffff
```

```
----- Tainted Mem -----
```

```
Addr: 804b008 -> ffffffff
```

```
Addr: 804b009 -> ffffffff
```

```
Addr: 804b00a -> ffffffff
```

```
Addr: 804b00b -> ffffffff
```

```
Addr: 804b00c -> ffffffff
```

```
Addr: 804b00d -> ffffffff
```

- Addresses with a taint value of -1 have merged paths from multiple bytes from the user supplied input
- Positive integers correspond to byte offsets in the original input
- Non-tainted addresses would hold a 0 in that field so offsets are base 1, not base 0

Addr:	dd540025	->	ffffffff
Addr:	dd540026	->	ffffffff
Addr:	dd540027	->	ffffffff
Addr:	dd540028	->	ffffffff
Addr:	dd540029	->	ffffffff
Addr:	dda0a000	->	1
Addr:	dda0a001	->	2
Addr:	dda0a002	->	3
Addr:	dda0a003	->	4
Addr:	dda0a004	->	5
Addr:	dda0a005	->	6
Addr:	dda0a006	->	7
Addr:	dda0a007	->	8
Addr:	dda0a008	->	9

- When an exception occurs the crash context is saved along with memory
- Each allocated page is written to disk for later automated crash analysis

```
edx v=0x0804b010, t=0x00000000
esi v=0x00000000, t=0x00000000
edi v=0x00000d80, t=0x00000000
ebp v=0xffffd238, t=0x00000000
esp v=0xffffd1f4, t=0x00000000
eflags v=0x00010202, t=0xffffffff
eip v=0xddba7cdf, t=0x00000000
0x01048000 - 0x017a3000 (7532KB)
0x017a6000 - 0x017a7000 (4KB)
0x017aa000 - 0x017d4000 (168KB)
0x017d5000 - 0x0181c000 (284KB)
0x08048000 - 0x0806c000 (144KB)
0xdd0fc000 - 0xdd1fe000 (1032KB)
0xdd20e000 - 0xdd20f000 (4KB)
0xdd239000 - 0xdd23a000 (4KB)
0xdd26d000 - 0xdf1e0000 (32204KB)
0xdf1e1000 - 0xdf5fb000 (4200KB)
0xdf5fc000 - 0xf7ffe000 (403464KB)
0xff7fe000 - 0xfffffe000 (8192KB)
Total snapshot size: 468205568B (457232KB) (446MB)
Received fatal signal b
```

```

.text:00048871
.text:00048872
.text:00048872 ; ===== SUBROUTINE =====
.text:00048872
.text:00048872
.text:00048872 foo          proc near          ; CODE XREF: nice_crash!p
.text:00048872
.text:00048872 arg_4          = dword ptr 8
.text:00048872
.text:00048872 mov     esi, [esp+arg_4]
.text:00048872 xor     eax, eax
.text:00048872 lodsb
;
; @context "R_EAX" = 0x0, 0, u32, wr @context "R_ESI" = 0x9cb0000, 0, u32, rd
; @context "EFLAGS" = 0x246, 0, u32, rd
; @context "mem[0x9cb0000]" = 0x41, 1, u8, rd
; label pc_0x8048878
; T_32t0:u32 = R_DFLAG:u32
; T_32t1:u32 = R_ESI:u32
; T_8t2:u8 = mem:?u32[T_32t1:u32, e_little]:u8
; R_EAX:u32 = R_EAX:u32 & 0xffffffff:u32 | pad:u32(T_8t2:u8)
; T_32t3:u32 = T_32t1:u32 + T_32t0:u32
; R_ESI:u32 = T_32t3:u32
;
.text:00048879 xor     edi, edi
.text:0004887B add     edi, eax
;
; @context "R_EDI" = 0x0, 0, u32, rw
; @context "R_EAX" = 0x41, 1, u32, rd @context "EFLAGS" = 0x246, 0, u32, wr
; label pc_0x804887b
; T_t1:u32 = R_EDI:u32
; T_t2:u32 = R_EAX:u32
; R_EDI:u32 = R_EDI:u32 + T_t2:u32
; R_CF:bool = R_EDI:u32 < T_t1:u32
; R_AF:bool = 0x10:u32 == (0x10:u32 & (R_EDI:u32 ^ T_t1:u32 ^ T_t2:u32))
; R_OF:bool = high:bool((T_t1:u32 ^ ~T_t2:u32) & (T_t1:u32 ^ R_EDI:u32))
; R_PF:bool =
; ~low:bool(R_EDI:u32 >> 7:u32 ^ R_EDI:u32 >> 6:u32 ^ R_EDI:u32 >> 5:u32 ^
; R_EDI:u32 >> 4:u32 ^ R_EDI:u32 >> 3:u32 ^ R_EDI:u32 >> 2:u32 ^
; R_EDI:u32 >> 1:u32 ^ R_EDI:u32)
; R_SF:bool = high:bool(R_EDI:u32)
; R_ZF:bool = 0:u32 == R_EDI:u32
;
;
; @context "R_EDI" = 0x41, 1, u32, rw
; @context "EFLAGS" = 0x206, 1, u32, wr
; label pc_0x804887d
; T_t:u32 = R_EDI:u32
; R_EDI:u32 = R_EDI:u32 - 0x30:u32
; R_CF:bool = T_t:u32 < 0x30:u32
; R_OF:bool = high:bool((T_t:u32 ^ 0x30:u32) & (T_t:u32 ^ R_EDI:u32))
; R_AF:bool = 0x10:u32 == (0x10:u32 & (R_EDI:u32 ^ T_t:u32 ^ 0x30:u32))
; R_PF:bool =
; ~low:bool(R_EDI:u32 >> 7:u32 ^ R_EDI:u32 >> 6:u32 ^ R_EDI:u32 >> 5:u32 ^
; R_EDI:u32 >> 4:u32 ^ R_EDI:u32 >> 3:u32 ^ R_EDI:u32 >> 2:u32 ^
; R_EDI:u32 >> 1:u32 ^ R_EDI:u32)
; R_SF:bool = high:bool(R_EDI:u32)
; R_ZF:bool = 0:u32 == R_EDI:u32
;
.text:0004887D sub     edi, 30h
;

```

Trace Slicing

Taint Slicing for Root Cause Analysis

- Now we have collected all instructions that interacted with user data, the values of that data for each instruction, and a snapshot of memory at a crash
- We will construct a dataflow dependency graph that holds all relationships between the instructions and the memory values through out the lifetime of the program
- Finally we will select a byte on that graph and find the path from exception back to the original input offset and value

Concept

- Methodology
 - Collect trace
 - Convert trace to BAP IL
 - Select location and value of interest (register or memory address)
 - Select direction of slice
 - Follow dependencies in desired direction to produce sub-graph

Concept

- Trace slicing finds the sub-graph of dependencies between two nodes
 - All nodes that influence or are influenced by specified node can be isolated
 - Reachability Problem
- Forward Slicing
 - Slice forward to determine instructions influenced by selected value
- Backward Slicing
 - Slice backward to locate the instructions influencing a value
 - Collect constraints to determine the degree of control over the value

Forward Slicing

- Slice forward to determine instructions influenced by a value

```
S = {v}
For each stmt in statements:
    If vars(stmt.rhs) ∩ S != ∅ then
        S := S ∪ {stmt.lhs}
Return S
```

stmt	S
<code>el_size, el_count, el_data = read()</code>	<code>{el_size}</code>
<code>total_size = el_size * el_count</code>	<code>{el_size, total_size}</code>
<code>buf = malloc(total_size)</code>	<code>{el_size, total_size}</code>
<code>while count < el_count</code>	<code>{el_size, total_size}</code>
<code>offset = count * el_size</code>	<code>{el_size, total_size, offset}</code>
<code>data_offset = el_data + offset</code>	<code>{el_size, total_size, offset, data_offset}</code>
<code>buf_offset = buf + offset</code>	<code>{el_size, total_size, offset, data_offset, buf_offset}</code>
<code>memcpy(buf_offset, data_offset, el_size)</code>	<code>{el_size, total_size, offset, data_offset, buf_offset}</code>

Backward Slicing

- Slice backward to locate the instructions influencing a value

```
S = {v}
For each stmt in reverse(statements):
    If {stmt.lhs} ∩ S != ∅ then
        S := S ∪ vars(stmt.rhs)
Return S
```

stmt	S
el_size, el_count, el_data = read()	{data_offset, el_data , offset, count, el_size}
total_size = el_size * el_count	{data_offset, el_data, offset, count, el_size}
buf = malloc(total_size)	{data_offset, el_data, offset, count, el_size}
while count < el_count	{data_offset, el_data, offset, count, el_size}
offset = count * el_size	{data_offset, el_data, offset , count, el_size}
data_offset = el_data + offset	{data_offset, el_data, offset }
buf_offset = buf + offset	{data_offset}
memcpy(buf_offset, data_offset , el_size)	{data_offset}

- To perform slicing on native assembly language we need to understand the semantics of every instruction
- This is tedious and error prone, especially when support for multiple architectures is desired
- A common solution for this problem is to use an intermediate assembly language that expands complex instructions to simplified RISC like architecture with all side effects explicit

Implementation Details

- BAP includes an intermediate assembly language definition called BIL
- BIL expands each native assembly instruction into a sequence of instructions representing each side effect
- Each instruction is easier to analyze and side effects are explicit
- We only have to handle assignments of the form *var := exp*

```
program ::= stmt*  
stmt ::= var := exp | jmp(exp) | cjmp(exp,exp,exp)  
        | halt(exp) | assert(exp) | label label_kind  
        | special(string)
```

Implementation Details

- BAP includes an intermediate assembly language definition called BIL
- BIL expands each native assembly instruction into a sequence of instructions representing each side effect
- Each instruction is easier to analyze and side effects are explicit
- We only have to handle assignments of the form *var := exp*

```
.text:08048887  mov edx, [edi+11223344h] ;  
.text:08048887      ; @context "R_EDX" = 0x1000, 0, u32, wr  
.text:08048887      ; @context "R_EDI" = 0x11, 1, u32, rd  
.text:08048887      ; @context "mem[0x11223355]" = 0x0, 0, u8, rd  
.text:08048887      ; @context "mem[0x11223356]" = 0x0, 0, u8, rd  
.text:08048887      ; @context "mem[0x11223357]" = 0x0, 0, u8, rd  
.text:08048887      ; @context "mem[0x11223358]" = 0x0, 0, u8, rd  
.text:08048887      ; label pc_0x8048887  
.text:08048887      ; R_EDX:u32 = mem:?u32[R_EDI:u32 + 0x11223344:u32, e_little]:u32
```

Slicing with moflow

- Get the prebuilt docker image:

```
vulndev@vulndev-x64 ~ $ sudo docker run -ti moflow/moflow-0.8
[sudo] password for vulndev:
root@d14e0cf39781:/moflow# cd slicer/
root@d14e0cf39781:/moflow/slicer# ls
common.cmi    ilprint.sh      readme.txt      slicer.cmo      triage.sh
common.cmo    makefile        run_demo.sh     slicer.ml
common.ml     motriage.ml     slicer          snapshot.format.txt
demo          prep-slice.sh   slicer.cmi      tests
root@d14e0cf39781:/moflow/slicer# ./run_demo.sh
```

Slicing with moflow

- First we will record the taint trace and then we will convert to the BAP intermediate language.
- We will use concrete substitution (concretization) to load the values that were in memory into the IL
- BAP is capable of doing fully static analysis but for our purposes of crash analysis we want to use the memory values

Slicing with moflow

- `root@d14e0cf39781:/moflow/slicer# sudo ../bap/pin/pin -t
../tracer/gentrace32.so -taint_indices -taint_files input.txt -snapshot-
file /tmp/demo.snapshot -o /tmp/demo.trace -- ./demo/demo tlv
demo/input.txt 2>/dev/null`
- `total_size: 65535, record_count 1111638594`
- `root@d14e0cf39781:/moflow/slicer# ../utils/iltrans -trace
/tmp/demo.trace -trace-concrete-subst -trace-dsa -pp-ast
/tmp/demo.trace.il`
- Concrete Substitution Run: Done! (0.412670 seconds)

Slicing with moflow

- We now have a file in /tmp/demo.trace.il that is plaintext. We could have output bjson, json, protobuf, or plaintext
- We see taint introduced at the top

```
/*ReadSyscall*/ @taint_intro 1, "file demo/input.txt", 0
@context "mem32[0xdda0a000]" = 0x41, 1, u8, wr
@taint_intro 2, "file demo/input.txt", 1
@context "mem32[0xdda0a001]" = 0x41, 2, u8, wr
@taint_intro 3, "file demo/input.txt", 2
@context "mem32[0xdda0a002]" = 0x41, 3, u8, wr
@taint_intro 4, "file demo/input.txt", 3
@context "mem32[0xdda0a003]" = 0x41, 4, u8, wr
@taint_intro 5, "file demo/input.txt", 4
@context "mem32[0xdda0a004]" = 0xff, 5, u8, wr
@taint_intro 6, "file demo/input.txt", 5
@context "mem32[0xdda0a005]" = 0xff, 6, u8, wr
@taint_intro 7, "file demo/input.txt", 6
@context "mem32[0xdda0a006]" = 0x0, 7, u8, wr
@taint_intro 8, "file demo/input.txt", 7
@context "mem32[0xdda0a007]" = 0x0, 8, u8, wr
@taint_intro 9, "file demo/input.txt", 8
@context "mem32[0xdda0a008]" = 0x42, 9, u8, wr
```

Slicing with moflow

- On the next slide we will skip down to the first instruction executed after the `read()` system call returns.
- Before the first instruction description we see there are variable names given to each byte that was read in.
- Each time a memory location is written to it will get a new variable name assigned. This is called a Static Single Assignment form and simplifies our slicing
- These are in the form `dsa_["mem" | "REG"]_[address]_1_[unique ID]`

```
dsa_mem_dda0affe_1_16372:u8 = symb_4095_8896:u8
dsa_mem_dda0afff_1_16373:u8 = symb_4096_8898:u8
addr 0xddae6318 @asm "movzx  eax,BYTE PTR [eax]" @tid "0"
    @context "R_EAX_32" = 0xdda0a000, 0, u32, wr
    @context "R_EAX_32" = 0xdda0a000, 0, u32, rd
    @context "mem32[0xdda0a000]" = 0x41, 1, u8, rd
label pc_0xddae6318
dsa_R_EAX_32_1_16374:u32 = 0xdda0a000:u32
dsa_R_EAX_32_1_16375:u32 = pad:u32(pad:u8(dsa_mem_dda0a000_1_12278:u8))
addr 0xddae5558 @asm "cmp     eax,0xffffffff" @tid "0"
    @context "R_EAX_32" = 0x41, 1, u32, rd
    @context "R_EFLAGS" = 0x286, 0, u32, wr
label pc_0xddae5558
dsa_R_ZF_1_16376:bool = false
dsa_R_AF_1_16377:bool = false
dsa_R_OF_1_16378:bool = false
dsa_R_SF_1_16379:bool = true
dsa_R_DF_1_16380:bool = false
dsa_R_CF_1_16381:bool = false
dsa_R_EFLAGS_1_16382:u32 = 0x286:u32
dsa_R_PF_1_16383:bool = true
```

Slicing with moflow

- That is a little cumbersome at first glance so let's break it down

```
addr 0xddae6318 @asm "movzx  eax,BYTE PTR [eax]" @tid "0"  
  @context "R_EAX_32" = 0xdda0a000, 0, u32, wr  
  @context "R_EAX_32" = 0xdda0a000, 0, u32, rd  
  @context "mem32[0xdda0a000]" = 0x41, 1, u8, rd
```

- This shows the values of EAX which was read, dereferenced, and written back in to EAX.
- The memory value in EAX that was dereferenced is shown on the last line. The field after the byte value is '1' meaning it is the first byte from our input file

Slicing with moflow

- Lets quickly take a look at the exception that we were tracing in the debugger

```
#define BUF_SIZE 1024
#define BIG 0x1000000

void read_file(char *fn, char *buffer, int size){
    FILE * pFile;
    size_t result;

    pFile = fopen (fn, "rb" );
    if (pFile==NULL) {fprintf(stderr, "Can't open %s", fn); exit (1);}

    result = fread (buffer,1,size,pFile);
    if (result != size)
        printf("read bytes %d < %d\n", result, size);

    fclose (pFile);
}
```

```
struct header {
    char magic[4];
    int total_size; // read av on large total_size
    int record_count;
};

struct record {
    int type; // read or subrecord
    int len; // write av on long len
    char val[1];
};

void test_tlv_triage(char *buf)
{
    char *newbuf, *r_copy;
    int count, offset;
    struct header *h = (struct header *)buf;
    struct record *r = (struct record *) (buf + sizeof(struct header));
```



```
if(memcmp(&h->magic, "AAAA", 4) != 0)
{
    printf("bad magic\n");
    return;
}
else
{
    printf("total_size: %d, record_count %d\n", h->total_size, h->record_count);
}
fflush(stdout);

newbuf = (char *)malloc(BUF_SIZE);
memcpy(newbuf, buf, h->total_size); // readAV if total_size > BUF_SIZE
```

```
count = 0;
offset = sizeof(struct header);
while(count < h->record_count)
{
    printf("record: type %d, len %d\n", r->type, r->len);
    if(r->type == 1)
    {
        memcpy(newbuf, r->val, r->len);
    }
    else if(r->type == 2)
    {
        memcpy(newbuf + (int)&r->val, r->val + 4, r->len);
    }

    offset += r->len;
    count++;
}

return;
```

```
vulndev@vulndev-x64 /vulndev/demo $ xxd input.txt | head -2
00000000: 4141 4141 ffff 0000 4242 4242 0100 0000  AAAA....BBBB....
00000010: ffff 0000 0000 0000 0000 0000 0000 0000  .....
vulndev@vulndev-x64 /vulndev/demo $ ./demo tlv input.txt
total_size: 65535, record_count 1111638594
Segmentation fault
```

```

[-----STACK-----]
00:0000 | esp 0xffffcbc4 ← 0x0
01:0004 | 0xffffcbc8 → 0x55752000 (_GLOBAL_OFFSET_TABLE_) ← 0x1acda8
02:0008 | 0xffffcbcc → 0x80488f5 (test_tlv_triage+158) ← mov dword ptr
[ebp - 0x1c], 0
03:000c | 0xffffcbd0 → 0x804b008 ← 0x41414141 ('AAAA')
04:0010 | 0xffffcbd4 → 0xffffcc2c ← 0x41414141 ('AAAA')
05:0014 | 0xffffcbd8 ← 0xffff
06:0018 | 0xffffcbdc → 0x80487e8 (read_file+155) → 0x8955c3c9 ← 0x8955c3c
9
07:001c | 0xffffcbe0 → 0x804b008 ← 0x41414141 ('AAAA')
[-----BACKTRACE-----]

```

```

▶ f 0 556daf06 __memcpy_ssse3_rep+1302
  f 1 80488f5 test_tlv_triage+158
  f 2 8048bb4 main+218
  f 3 555beaf3 __libc_start_main+243

```

Program received signal SIGSEGV (fault address 0xffffe000)

- We crashed in a memcpy. We know that if the user controls any of the arguments to a memcpy this could be a potentially exploitable bug
- We are not interested in the internals of libc, we can see that memcpy was called from our main image, so lets just get the tainted instructions in our main image

- One of these instructions required some sanity checking before the user controlled operands were passed as arguments to memcpy

```
root@d14e0cf39781:/moflow/slicer# grep "addr 0x804" /tmp/demo.trace.il
```

```
addr 0x80488a0 @asm "mov    edx,DWORD PTR [eax+0x8]" @tid "0"  
addr 0x80488a6 @asm "mov    eax,DWORD PTR [eax+0x4]" @tid "0"  
addr 0x80488a9 @asm "mov    DWORD PTR [esp+0x8],edx" @tid "0"  
addr 0x80488ad @asm "mov    DWORD PTR [esp+0x4],eax" @tid "0"  
addr 0x80488dc @asm "mov    eax,DWORD PTR [eax+0x4]" @tid "0"  
addr 0x80488df @asm "mov    DWORD PTR [esp+0x8],eax" @tid "0"  
addr 0x80488e3 @asm "mov    eax,DWORD PTR [ebp+0x8]" @tid "0"  
addr 0x80488e6 @asm "mov    DWORD PTR [esp+0x4],eax" @tid "0"
```

- We could run the tool three times, slicing back on any byte from each of the three memcpy parameters. In the last instructions executed before calling memcpy we see that the instructions that were setting the “src” and “size” parameters to memcpy were tainted.

```
root@d14e0cf39781:/moflow/slicer# grep "addr 0x804" /tmp/demo.trace.il
```

```
addr 0x80488a0 @asm "mov    edx,DWORD PTR [eax+0x8]" @tid "0"  
addr 0x80488a6 @asm "mov    eax,DWORD PTR [eax+0x4]" @tid "0"  
addr 0x80488a9 @asm "mov    DWORD PTR [esp+0x8],edx" @tid "0"  
addr 0x80488ad @asm "mov    DWORD PTR [esp+0x4],eax" @tid "0"  
addr 0x80488dc @asm "mov    eax,DWORD PTR [eax+0x4]" @tid "0"  
addr 0x80488df @asm "mov    DWORD PTR [esp+0x8],eax" @tid "0"  
addr 0x80488e3 @asm "mov    eax,DWORD PTR [ebp+0x8]" @tid "0"  
addr 0x80488e6 @asm "mov    DWORD PTR [esp+0x4],eax" @tid "0"
```

- EAX is being moved to the stack, so let's see the instruction before the mov to ESP+8 and grab the variable name of a memory byte that EAX was pointing to

```

root@d14e0cf39781:/moflow/slicer# tac /tmp/demo.trace.il | grep -B13 -m1 "esp+0x8"
addr 0x80488e3 @asm "mov      eax,DWORD PTR [ebp+0x8]" @tid "0"
dsa_mem_ffffd20b_1_24602:u8 = low:u8(dsa_R_EAX_32_1_24597:u32 >> 0x18:u32)
dsa_mem_ffffd20a_1_24601:u8 = low:u8(dsa_R_EAX_32_1_24597:u32 >> 0x10:u32)
dsa_mem_ffffd209_1_24600:u8 = low:u8(dsa_R_EAX_32_1_24597:u32 >> 8:u32)
dsa_mem_ffffd208_1_24599:u8 = low:u8(dsa_R_EAX_32_1_24597:u32)
dsa_R_ESP_32_1_24598:u32 = 0xffffd200:u32
label pc_0x80488df
  @context "mem32[0xffffd20b]" = 0x42, -1, u8, wr
  @context "mem32[0xffffd20a]" = 0x42, -1, u8, wr
  @context "mem32[0xffffd209]" = 0x42, -1, u8, wr
  @context "mem32[0xffffd208]" = 0x42, -1, u8, wr
  @context "R_EAX_32" = 0xffff, -1, u32, rd
  @context "R_ESP_32" = 0xffffd200, 0, u32, rd
addr 0x80488df @asm "mov      DWORD PTR [esp+0x8],eax" @tid "0"

```


- The dsa_mem variable names are the expected input for our slicer so we can pick one and now get a slice of only the instructions that touched that byte of data

```
root@d14e0cf39781:/moflow/slicer# ./slicer -il /tmp/demo.trace.il -var dsa_mem_ffffd20b
_1_24602 2>/dev/null | grep addr
addr 0xddaf3506 @asm "rep movs DWORD PTR es:[edi],DWORD PTR ds:[esi]"
addr 0x80488dc @asm "mov     eax,DWORD PTR [eax+0x4]" @tid "0"
addr 0x80488df @asm "mov     DWORD PTR [esp+0x8],eax" @tid "0"
```

- We see the path from libc copying the file from fread into a buffer and then copying values from that buffer to the length parameter to memcpy()

- Now we examine the operands to that first instruction to see the byte offsets in the file. They are shown below as byte 5, 6, 7, and 8 which are base 1. We can see the value was 65535 which was too large and unchecked, resulting in a ReadAV

```
root@d14e0cf39781:/moflow/slicer# ./slicer -il /tmp/demo.trace.il -var  
dsa_mem_ffffd20b_1_24602 2>/dev/null | grep -A10 0xddaf3506  
addr 0xddaf3506 @asm "rep movs DWORD PTR es:[edi],DWORD PTR ds:[esi]"  
@tid "0" @context "R_EDI_32" = 0xffffd260, 0, u32, rd  
@context "R_ESI_32" = 0xdda0a004, 0, u32, rd  
@context "R_ECX_32" = 0xff, -1, u32, rw  
@context "R_EFLAGS" = 0x246, 0, u32, rd  
@context "mem32[0xdda0a004]" = 0xff, 5, u8, rd  
@context "mem32[0xdda0a005]" = 0xff, 6, u8, rd  
@context "mem32[0xdda0a006]" = 0x0, 7, u8, rd  
@context "mem32[0xdda0a007]" = 0x0, 8, u8, rd
```

Concolic Execution Driven Fuzzing

Automated Test Generation

- Goal: Exercise target program to achieve full coverage of all possible states influenced by external input
- Code graph reachability exercise
- Input interaction with conditional logic in program code determines what states you can reach

Concolic Testing

- For anything beyond string grouping algorithms, direct instrumentation of the code and observation of interaction between data and conditional logic is required
- Early academic work in this area:
 - DART: Directed Automated Random Testing
 - 2005 - Patrice Godefroid, et al
 - CUTE: a concolic unit testing engine for C
 - 2005 - Sen, Koushik
 - EXE: Automatically Generating Inputs of Death
 - 2006 -Dawson Engler

Concolic Test Generation: Core Concepts

Code Coverage & Taint Analysis

- Code Coverage
 - Analysis of program runtime to determine execution flow
 - Collect the sequence of execution of basic blocks and branch edges
- Several approaches
 - Native debugger API
 - CPU Branch Interrupts
 - Static binary rewriting
 - Dynamic binary instrumentation

Code Coverage & Taint Analysis

- Taint Analysis
 - Analysis of program runtime to determine data flow from external input throughout memory
 - Monitor each instruction for propagation of user controlled input from source operands to destination operands
 - Dependency tree is generated according to tainted data flows in memory or CPU registers
 - Taint analysis is imperfect – propagation rules must dictate the level of inferred dataflow that is propagated

Dynamic Binary Instrumentation

- JIT modification of binary code
 - As new code blocks are visited or modules are loaded, an analysis phase disassembles the binary to identify code structure
 - Instructions may be inserted at arbitrary locations around or within the disassembled target binary
 - Modified code is cached and referenced instead of original binary
- Skips some problems with static binary rewriting and maintains runtime state for conditional instrumentation

Symbolic Execution

- Symbolic execution involves computation of a mathematical expression that represents the logic within a program.
- It can be thought of as an algebra designed to express computation.

```
void test(char *buf)
{
    int n = 0;
    if(buf[0] == 'b') n++;
    if(buf[1] == 'a') n++;
    if(buf[2] == 'd') n++;
    if(buf[3] == '!') n++;
    if(n==4) {
        crash();
    }
}
```

```
(declare-const buf (Array Int Int))
(declare-fun test () Int)
(declare-const n Int)
(assert (= n 0))
(ite (= (select buf 0) 98) (+ n 1) 0)
(ite (= (select buf 1) 97) (+ n 1) 0)
(ite (= (select buf 2) 100) (+ n 1) 0)
(ite (= (select buf 3) 92) (+ n 1) 0)
(assert (= n 4))
(check-sat)
(get-model)
```

Symbolic Execution

- Symbolic execution involves computation of a mathematical expression that represents the logic within a program.
- It can be thought of as an algebra designed to express computation.

```
void condition(int x)
{
    int ret = 0;
    if (x > 50)
        ret =
1;
    else
        ret =
2;
    return ret
}
```

```
(declare-fun condition () Int)
(declare-const ret Int)
(declare-const x Int)
(assert (=> (>= x 50) (= ret 1)))
(assert (=> (< x 50) (= ret 2)))
(assert (= ret 1))
(check-sat)
(get-model)
---
sat
(model
  (define-fun x () Int 50)
  (define-fun ret () Int 1)
)
```

Symbolic Execution

- In my previous research I used Symbolic Execution to emulate forward from a crash to determine exploitability

```
void test_motriage(unsigned int *buf)
{
    unsigned int b,x,y;

    b = buf[0];
    x = buf[b+0x11223344];
    y = buf[x];
    exploit_me(1, x, y);
}
```

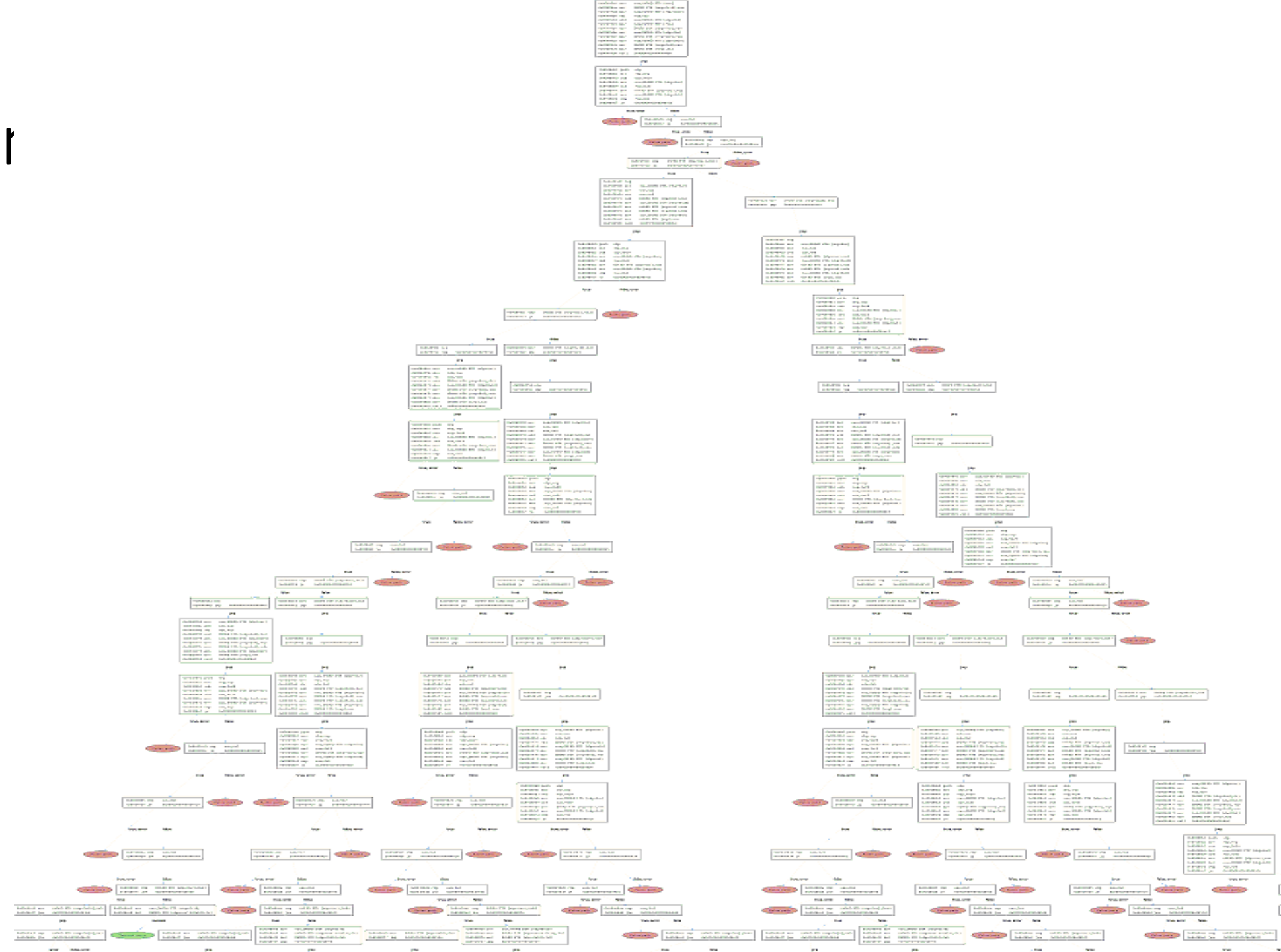
Symbolic Execution

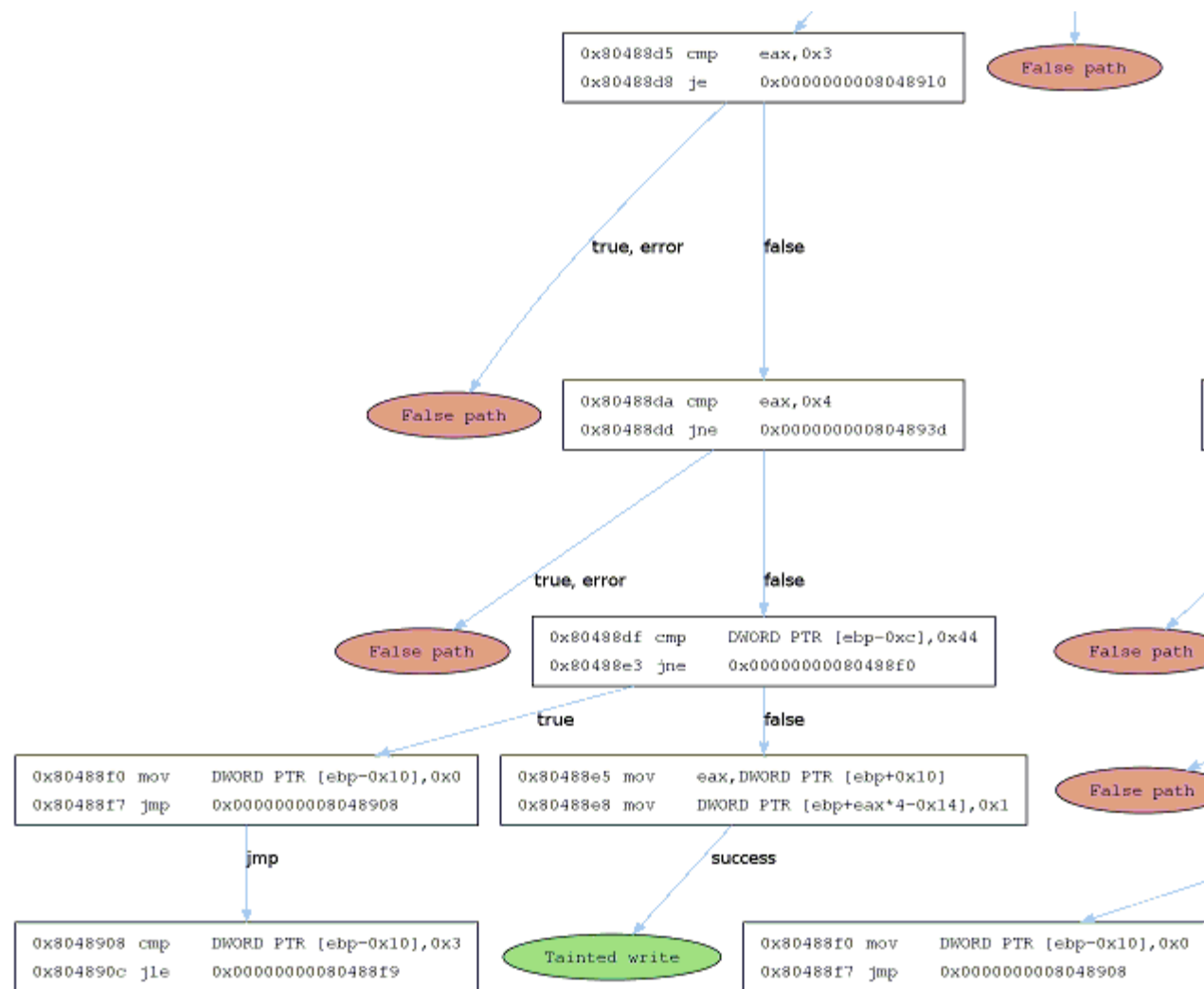
- In my previous research I used Symbolic Execution to emulate forward from a crash to determine exploitability

```
void exploit_me
(int depth,
 unsigned int x,
 unsigned int y)
{
    int stack[1];
    int b, i;
    b = x & 0xff;
    switch(depth) {
    ...
    }
    exploit_me(++depth, x>>8, y);
}
```

```
case 4:
    if(b == 0x44)
        stack[y] = 1;
    return;
case 3:
    if(b != 0x33) y = 0;
    break;
case 2:
    if(b != 0x22) y = 0;
    break;
case 1:
    if(b != 0x11) y = 0;
    break;
default:
    assert(0);
```

Syr





Constraint Generation

- Comparisons are done on values to determine which branch of code to take:

```
if (a > b):  
    block1  
else:  
    block2
```

- We observe these constraints to determine what data value ranges allow execution in different paths
- A code path is determined by collecting a series of these constraints which determines the execution flow of the program

Constraint Generation

- Against binary targets we need to track flags and evaluate the dependent comparison before a jump

```
0x080483d4 <+0>:      push    %ebp
0x080483d5 <+1>:      mov     %esp,%ebp
0x080483d7 <+3>:      and     $0xffffffff0,%esp
0x080483da <+6>:      sub     $0x10,%esp
0x080483dd <+9>:      cmpl   $0x1,0x8(%ebp)
0x080483e1 <+13>:     jle     0x80483f1 <main+29>
0x080483e3 <+15>:     movl   $0x80484d0,(%esp)
0x080483ea <+22>:     call   0x80482f0 <puts@plt>
0x080483ef <+27>:     jmp     0x80483f2 <main+30>
0x080483f1 <+29>:     nop
0x080483f2 <+30>:     leave
0x080483f3 <+31>:     ret
```

- This may be done manually or through the use of an IR

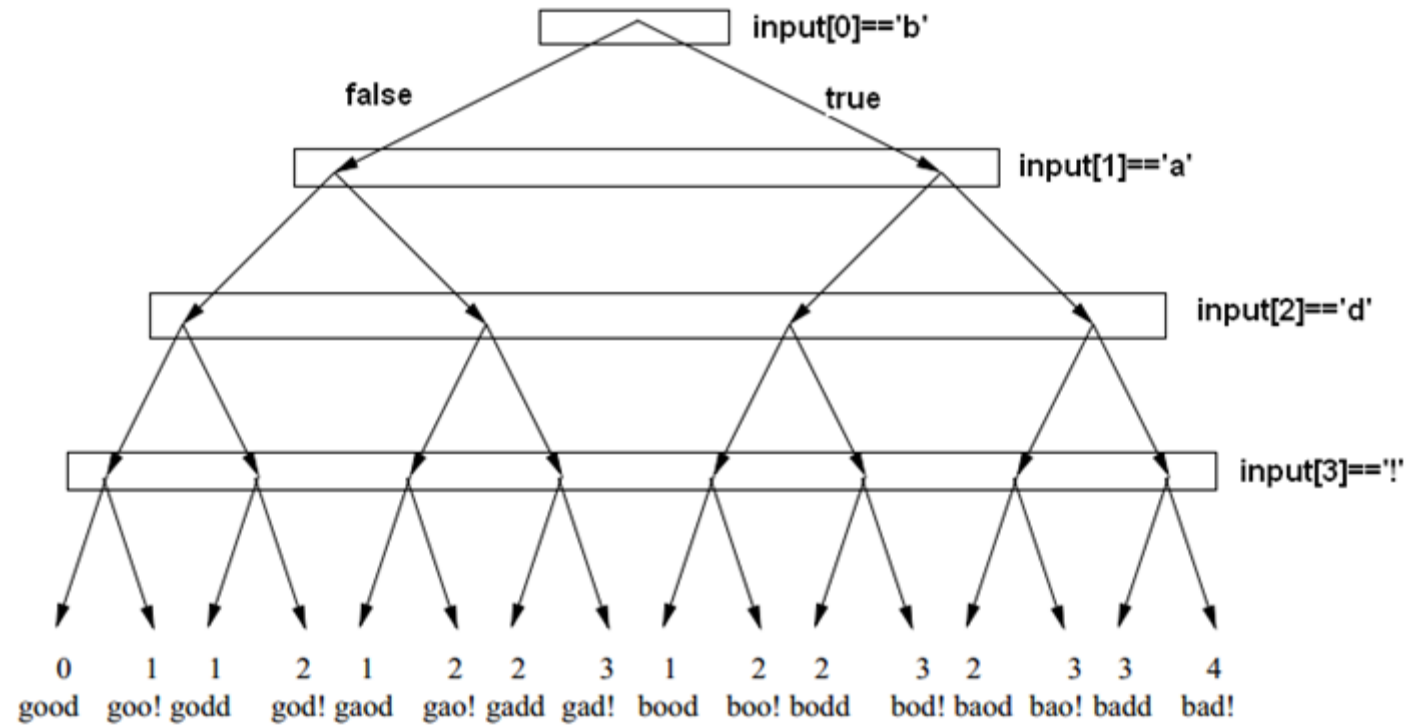
Constraint Solving

- A formula representing the code path logic is generated in a format acceptable to a symbolic execution engine
- To explore alternate paths, we invert the conditional logic of the last branch and allow the solver to generate an example that would match the inverted conditional logic
- Iterative use of this algorithm allows us to explore a complete program graph

Test Generation

- Input: 'bad?'
- Formula generated by symbolic execution:
 - $\Phi := (i_0 = 'b') \ \&\& \ (i_1 = 'a') \ \&\& \ (i_2 = 'd') \ \&\& \ (i_3 \neq '!')$
- New formulas:
 - $\Phi_0 := (i_0 = 'b') \ \&\& \ (i_1 = 'a') \ \&\& \ (i_2 = 'd') \ \&\& \ (i_3 = '!')$
 - $\Phi_1 := (i_0 = 'b') \ \&\& \ (i_1 = 'a') \ \&\& \ (i_2 \neq 'd') \ \&\& \ (i_3 \neq '!')$
 - $\Phi_2 := (i_0 = 'b') \ \&\& \ (i_1 \neq 'a') \ \&\& \ (i_2 = 'd') \ \&\& \ (i_3 \neq '!')$
 - $\Phi_3 := (i_0 \neq 'b') \ \&\& \ (i_1 = 'a') \ \&\& \ (i_2 = 'd') \ \&\& \ (i_3 \neq '!')$

Test Generation



Microsoft SAGE

Implementation

Tester	• AppVerifier crash harness
Tracer	• iDNA DBI Framework
CoverageCollector	• Coverage analysis of iDNA trace using Nirvana
SymbolicExecutor	• X86->SMT translator and constraint collector over iDNA trace using TruScan
Disolver	• Z3 constraint solver

Optimizations

- Generational Search vs DFS
 - DFS or BFS would negate only one of the branches
 - Generational search negates each condition and solves for each, generating many new inputs per symbolic execution phase instead of just one
- Constraint Optimization
 - Constraint Elimination - reduces the size of constraint solver queries by removing the constraints which do not share symbolic variables with the negated constraint
 - Local constraint Caching - skips a constraint if it has already been added to the path constraint
 - Flip count limit - establishes the maximum number of times a constraint generated from a particular program instruction can be flipped
 - Constraint Subsumption - tracks constraints dominated by a specific branch, skips identical constraints generated from the same instruction location

Results

- Thousands of crashes found in the Windows 7 and Office products – 1/3 of all file fuzzing bugs since 2007
- Lessons Learned
 - Vulnerabilities discovered are usually at shallow code depths
 - Symbolic Execution state is limited so wrappers need to be developed for library code
 - A small number of generations typically find the majority of vulnerabilities

Moflow::FuzzFlow

Implementation

Tracer

- Modified BAP pintool to collect memory dumps, coverage information, input stream names. Detects exceptions as well

Symbolic Executor

- Modification to BAP that supports converting BAP IL to SMTLIB formulas

SMT Solver

- We use z3 or STP to solve generated formulas

FuzzFlow Logic

- Custom tool built on top of BAP that glues all components together and implements the exploration algorithm

Limitations

- Tracer
 - Taint tracer from BAP is not optimized
 - For this application, inputs over a few kB are problematic
 - PIN is unable to flush single basic block hooks from code cache for code coverage hit trace
- Symbolic Execution
 - Slow conversion from BIL to SMTLIB on big traces
- FuzzFlow
 - Libraries need to be wrapped directly
 - We lack most of the optimizations in SAGE such as constraint subsumption

Does It Blend?

```
int main(int argc, char *argv[])
{
    char buf[500];
    size_t count;
    fd = open(argv[1], O_RDONLY);
    if(fd == -1) {
        perror("open");
        exit(-1);
    }
    count = read(fd, buf, 500);
    if(count == -1) {
        perror("read");
        exit(-1);
    }
    close(fd);
    test(buf);
    return 0;
}
```

```
void crash(){
    int i;
    // Add some basic blocks
    for(i=0;i<10;i++){
        i += 1;
    }
    *(int*)NULL = 0;
}

void test(char * buf)
{
    int n=0;
    if(buf[0] == 'b') n++;
    if(buf[1] == 'a') n++;
    if(buf[2] == 'd') n++;
    if(buf[3] == '!') n++;
    if(n==4){
        crash();
    }
}
```

Does It Blend?

```
moflow@ubuntu:~/moflow-bap-0.7/custom_utils/egas$ ./egas -app test/bof1 -seed test/input.txt
Starting program
Thread 0 starting
Opening tainted file: samples/13.sol
Tainting 5 bytes from read at bffafe30
buffer_size: 5, requested length: 5
Taint introduction #0. @bffafe30/5 bytes: file samples/13.sol
adding new mapping from file samples/13.sol to 0 on taint num 1
adding new mapping from file samples/13.sol to 1 on taint num 2
adding new mapping from file samples/13.sol to 2 on taint num 3
adding new mapping from file samples/13.sol to 3 on taint num 4
adding new mapping from file samples/13.sol to 4 on taint num 5
Activating taint analysis
CRASH! Sample: samples/13.sol saved as crashes/2014-06-20_22:40:10_13.crash
-----STATS-----
%          total      count      desc
68%        13s        9         taint tracing the target (produces .bpt)
16%         3s        14        gathering coverage info
5%          1s         9         symbolic execution
0%          0s         0         .bpt concretization
0%          0s        13         solver interaction
11%         2s         1         unaccounted
-----
elapsed: 19.000000
```

Real World Vulnerability Discovery

```
moflow@ubuntu:~/moflow-bap-0.7/custom_utils/egas$ ./egas -app /home/moflow/graphite2-1.2.3/tests/comparerenderer/comparerenderer -seed /home/moflow/graphite2-1.2.3/tests/fonts/tiny.ttf -fmt "-t /home/moflow/graphite2-1.2.3/tests/texts/udhr_nep.txt -s 12 -f %s -n"
```

```
Breakpoint 1, _IO_fread (buf=0x0, size=1, count=3758096384, fp=0x8053230) at iofread.c:37  
37 in iofread.c
```

```
(gdb) bt
```

```
#0  _IO_fread (buf=0x0, size=1, count=3758096384, fp=0x8053230) at iofread.c:37  
#1  0x4003a8ca in graphite2::FileFace::get_table_fn(void const*, unsigned int, unsigned int*)  
    ()  
    from /home/moflow/graphite2-1.2.3/src/libgraphite2.so.3  
#2  0x4002e8e5 in graphite2::Face::Table::Table(graphite2::Face const&, graphite2::TtfUtil::Tag) ()  
    from /home/moflow/graphite2-1.2.3/src/libgraphite2.so.3  
#3  0x4002858a in (anonymous namespace)::load_face(graphite2::Face&, unsigned int) ()  
    from /home/moflow/graphite2-1.2.3/src/libgraphite2.so.3  
#4  0x40028695 in gr_make_face_with_ops () from /home/moflow/graphite2-1.2.3/src/libgraphite2.so.3  
#5  0x40028aac in gr_make_file_face () from /home/moflow/graphite2-1.2.3/src/libgraphite2.so.3  
#6  0x0804d56d in Gr2Face::Gr2Face(char const*, int, std::string const&, bool) ()  
#7  0x0804b664 in main ()
```

Real World Vulnerability Discovery

```
const void *FileFace::get_table_fn(const void* appFaceHandle, unsigned int name, size_t *len)
{
    if (appFaceHandle == 0) return 0;
    const FileFace & file_face = *static_cast<const FileFace *>(appFaceHandle);
    void *tbl;
    size_t tbl_offset, tbl_len;
    if (!TtfUtil::GetTableInfo(name, file_face._header_tbl,
                               file_face._table_dir, tbl_offset, tbl_len))
        return 0;

    if (tbl_offset + tbl_len > file_face._file_len
        || fseek(file_face._file, tbl_offset, SEEK_SET) != 0)
        return 0;

    tbl = malloc(tbl_len);
    if (fread(tbl, 1, tbl_len, file_face._file) != tbl_len)
    {
        free(tbl);
        return 0;
    }

    if (len) *len = tbl_len;
    return tbl;
}
```

Exercise

- For this exercise, you will use moflow tools v0.8 and map in /vulnDev directory from host to /vulnDev
 - `docker run -v /vulnDev:/vulnDev -ti moflow/moflow-0.8`
- I have already prepared demo scripts and readme's in the docker
- For test case generation use the /moflow/egas folder and `./run_demo.sh`
- Now copy `run_demo.sh` to a new file and modify it for your target library

Browser Fuzzing w/ Domato

Domato - a useful grammar fuzzer

- Domato - Ivan Fratric / Google
 - Ivan is the author of both WinAFL and Domato
 - <https://github.com/google/domato>
- Domato has three components
 - grammar.py - Generic BNF parser and generative fuzzer
 - *.txt, *.html - HTML/CSS/js grammar and .html template
 - generator.py - Test case generator for DOM fuzzing

Domato - a useful grammar fuzzer

- Usage

```
python generator.py --output_dir <out> --no_of_files <count>
```

- `python generator.py --output_dir out --no_of_files 3`

Running on ClusterFuzz

Output directory: out

Number of samples: 3

Writing a sample to out\fuzz-0.html

Writing a sample to out\fuzz-1.html

Writing a sample to out\fuzz-2.html

Domato - a useful grammar fuzzer

Directory of C:\vulnDev\domato\out

08/30/2018	07:40 PM	<DIR>	.
08/30/2018	07:40 PM	<DIR>	..
08/30/2018	07:40 PM	455,255	fuzz-0.html
08/30/2018	07:40 PM	463,901	fuzz-1.html
08/30/2018	07:40 PM	460,684	fuzz-2.html

Domato - a useful grammar fuzzer

- Domato uses a modified BNF style grammar

`<cssproperty_z-index> = <fuzzint>`

`<cssproperty_z-index> = auto`

`<cssproperty_z-index> = inherit`

`<cssproperty_zoom> = <percentage>%`

`<cssproperty_zoom> = <fuzzint>`

`<cssproperty_zoom> = <float>`

`<cssproperty_zoom> = auto`

`<cssproperty_zoom> = reset`

Domato Grammar

- To inject lines of code into the grammar, a special syntax is used

!begin lines

```
<new element> = document.getElementById("<string min=97 max=122>");
```

```
<element>.doSomething();
```

!end lines

- This will expand types as well as generate new variable names

Domato Grammar

- To inject lines of code into the **generator**, another syntax is used

```
!begin function createbody  
  n = int(context['size'])  
  ret_val = 'a' * n  
!end function
```

```
<body> = <call function=createbody>
```

Adding Coverage to Grammar

- We have been talking about code coverage which amounts to visiting edges on a directed graph.
 - Grammars are graphs too!
- Domato generating a complete grammar blob to inject into the template
 - Grammar rules are selected at random and expanded.
- Large grammars have a lot of edges so we can optimize the generation of a diverse set of edges rather than relying on random
 - We can ensure that the grammar rules selected to expand symbols are unique or transitions between rules are unique with an AFL style bloom filter

Adding Coverage to Grammar

- AFL_MAP Bloom Filter
 - AFL tracks transitions between two adjacent nodes on a graph
 - Edge tracking is much more effective than simple block
 - The tracking must be space efficient and CPU cache aware
 - Every edge will be logged, so performance considerations are critical
 - CPUs must translate virtual addresses to linear addresses in the TLB
 - Consumer SKUs of Intel CPUs have 64kb L1 cache

Adding Coverage to Grammar

- Requirements
 - Assign a unique ID to each rule
 - Add a bloom filter to track edges between grammar rules
 - During rule selection track how many times an edge is transitioned
 - Select one of the expansion rules that has been visited least to create a DFS on the grammar

Logging Grammar Coverage

- Generators for each expansion rule are in the creators list
- Here we track the sequence of rules that are generated by hashing our ID with prev_id

```
for c in creators:
    bitmap_idx = (c['uid'] ^ self._prev_id) % MAP_SIZE
    if bitmap_idx not in self._cov:
        self._cov[bitmap_idx] = 1
        self._prev_id = c['uid'] >> 1
        return c
    else:
        hits.append(self._cov[bitmap_idx])
```

Rule Expansion Candidates

- Here we select the rules that match the minimum hit count

```
min_hit_count = min(hits)
    for i in [index for index, element in enumerate(hits) if min_hit_count ==
element]:
        less_visited_creators.append(creators[i])
```

Expanding Rules

- Now we select one of those rules to expand the current symbol

```
idx = random.randint(0, len(less_visited_creators) - 1)
curr_id = less_visited_creators[idx]['uid']
self._cov[(self._prev_id ^ curr_id) % MAP_SIZE] += 1
self._prev_id = curr_id >> 1
return less_visited_creators[idx]
```

Cumulative Distribution Functions

- Cumulative Distribution Functions
- Domato already has the concept of weights applied to rules, though they do not exist in the default grammars
- Next steps are to dynamically generate CDFs as usage patterns evolve.

Selfhosting Generator

- Domato's generator writes files to disk by default and exits
- We are not storing our bloom filter to disk
- Lets make generator.py a webserver

```
def main():  
    cherrypy.quickstart(DomFuzzer(os.path.dirname(__file__)))
```

Selfhosting Generator

- A top level function reads in the grammar files and parses them with grammar.py

```
def generate_samples(grammar_dir, outfiles):  
    """Generates a set of samples and writes them to the output files.  
    Args:  
        grammar_dir: directory to load grammar files from.  
        outfiles: A list of output filenames.  
    """
```


Selfhosting Generator

... continued..

```
f = open(os.path.join(grammar_dir, 'template.html'))
```

```
template = f.read()
```

```
f.close()
```

```
htmlgrammar = Grammar()
```

```
err = htmlgrammar.parse_from_file(os.path.join(grammar_dir, 'html.txt'))
```

...

```
self.generate_new_sample(template, htmlgrammar, cssgrammar, jsgrammar)
```

Selfhosting Generator

- We replace that function with our server's default web handler

```
class DomFuzzer(object):
    @cherry.py.expose
    def default(self,*args,**kwargs):
        outfile = 'out/%d.html' % time.time()
        grammar_dir = os.path.dirname(__file__)
        f = open(os.path.join(grammar_dir, 'template.html'))
        template = f.read()
        f.close()

        ...
        return self.generate_new_sample(template, htmlgrammar, cssgrammar,
jsgrammar)
```

Selfhosting Generator

- Generator's main function takes grammar objects and injects them into a html template

```
def generate_new_sample(template, htmlgrammar, cssgrammar, jsgrammar):  
    """Parses grammar rules from string.  
    Args:  
        template: A template string.  
        htmlgrammar: Grammar for generating HTML code.  
        cssgrammar: Grammar for generating CSS code.  
        jsgrammar: Grammar for generating JS code.  
    Returns:  
        A string containing sample data.  
    """
```

Selfhosting Generator

- Generator's main function takes grammar objects and injects them into a html template

```
def generate_new_sample(template, htmlgrammar, cssgrammar, jsgrammar):  
    """Parses grammar rules from string.  
    Args:  
        template: A template string.  
        htmlgrammar: Grammar for generating HTML code.  
        cssgrammar: Grammar for generating CSS code.  
        jsgrammar: Grammar for generating JS code.  
    Returns:  
        A string containing sample data.  
    """
```

Selfhosting Generator

- Generator's main function takes grammar objects, sets context, expands !function rules
- Deserialize the grammar tree and inject them into a html template

```
self.generate_html_elements(htmlctx, _N_ADDITIONAL_HTMLVARS)  
result = result.replace('<cssfuzzer>', css)  
result = result.replace('<htmlfuzzer>', html)
```

- Further javascript is generated based on the HTML objects generated and the result is returned

Day 3 Wrap Up Discussion